

A MENTORSHIP KIT

Systems, and How to Design Them

Seven words, five kinds of solution, six kits, and two systems designed end to end.



A complex system that works is invariably found to have evolved from a simple system that worked.

JOHN GALL, 1975 — THE SENTENCE MOST SYSTEM DESIGN GOES WRONG BY IGNORING

WHY THIS IS HARD

Nobody is stuck on the technology. They are stuck on the thinking.

No vocabulary

Without words for boundary and invariant, the discussion becomes a list of product names.

No frame

"Design Instagram" has no answer until someone says which solution is wanted.

No reference

Advice arrives as folklore, one anecdote at a time, never as a kit you carry.

KEY INSIGHT

A small vocabulary, one framing question, and six kits you keep for a career.

What we build together, in five parts.

- 01 The vocabulary — system, process, model, and five more
- 02 Model your day — the method on a problem you know
- 03 Solution types — which answer is being asked for
- 04 Six kits — data, compute, network, scale, reliability, security
- 05 Two systems — a photo feed and a chat model

01

PART ONE

Seven words do most of the work.

Stop calling everything "the system" and the design gets easier.

Four words separate what you are building from the world it sits in.

01	System	Parts joined by relationships.	What does the whole do?
02	Boundary	The line between in and out.	What is mine to fix?
03	Environment	What acts on you from outside.	What can I not control?
04	Process	A repeatable transformation.	What turns input into output?

Three more words are what you actually reason with.

You never handle the system itself. You handle a drawing of it, its limits, and its promises.

01	Model	A chosen simplification.	What did I leave out?
02	Constraint	A limit you design within.	What is not negotiable?
03	Invariant	A claim that must stay true.	What would mean broken?

A system is parts, connected, doing something together.

*What is it made of, and what is it
for?*

Parts, the relationships between them,
and a purpose no part serves alone.
Remove one and you have a pile.

YOU KNOW YOU'RE HERE WHEN

You can name a behavior the whole has that no single part has.

THE PARTS ARE THE EASY HALF

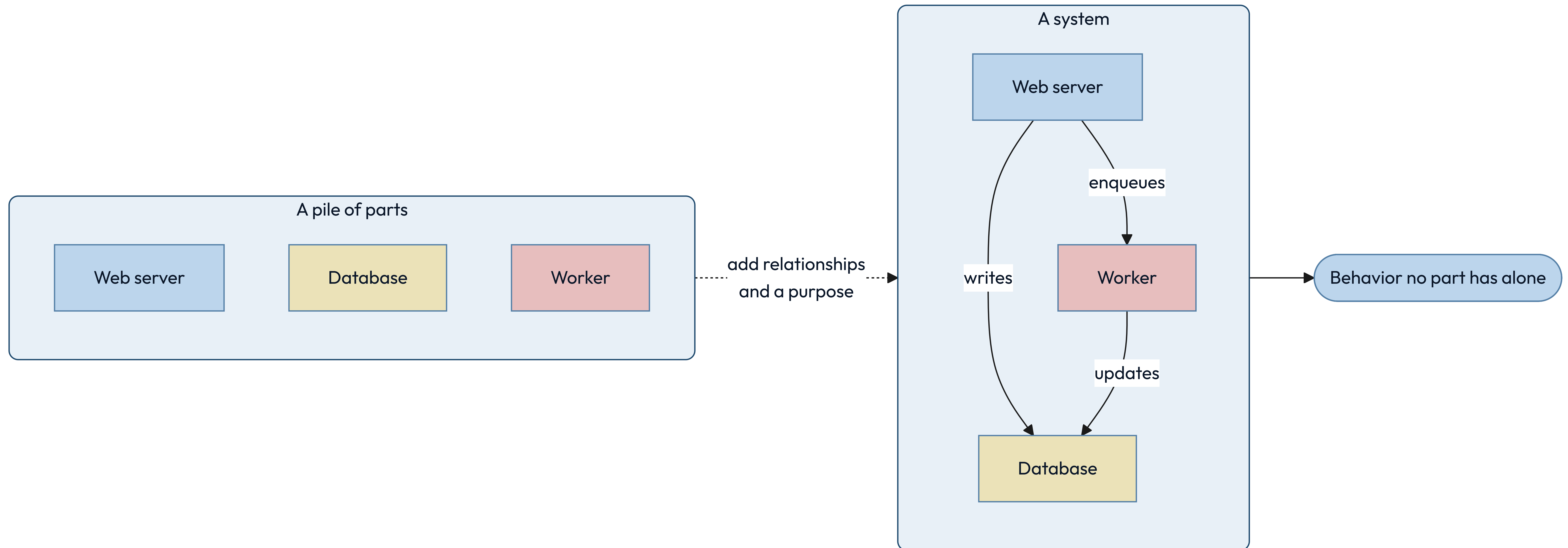
Servers, queues, people, a database, a
payment provider. You can list them.

THE RELATIONSHIPS ARE THE SYSTEM

Same parts, different wiring,
completely different behavior.

THE DEFINITION, DRAWN

The same parts, wired differently, are a different system.



The boundary is the line you are accountable for.

What is mine, and what is merely near me?

Inside, you change things. Outside, you make requests and handle refusals.

Drawing that line is the first real design decision.

YOU KNOW YOU'RE HERE WHEN

You can say which failures page you at 3am and which ones do not.

INSIDE IS WHAT YOU CHANGE

Your code, your schema, your deploys, your rotation.

OUTSIDE IS WHAT YOU NEGOTIATE

A payment provider, an identity service, a customer's browser.

The environment acts on you and never asks permission.

What arrives that I did not choose?

Traffic spikes, hostile users, dying disks, changing laws, a partner shipping a breaking version on a Friday. None of it is yours.

YOU KNOW YOU'RE HERE WHEN

You have written down what arrives, not what you plan to build.

IT IS NOT THE SAME AS LOAD

Regulation, competitors and staff turnover are environment too.

YOU DESIGN FOR IT, NOT AGAINST IT

You cannot stop a partition. You can decide what happens during one.

A process is a repeatable transformation that takes time.

What turns this input into that output?

A system is a noun; a process is a verb.
Each one has inputs, a transformation, outputs, a rate, and state it leaves behind.

YOU KNOW YOU'RE HERE WHEN

Someone else could run the sequence without asking you a question.

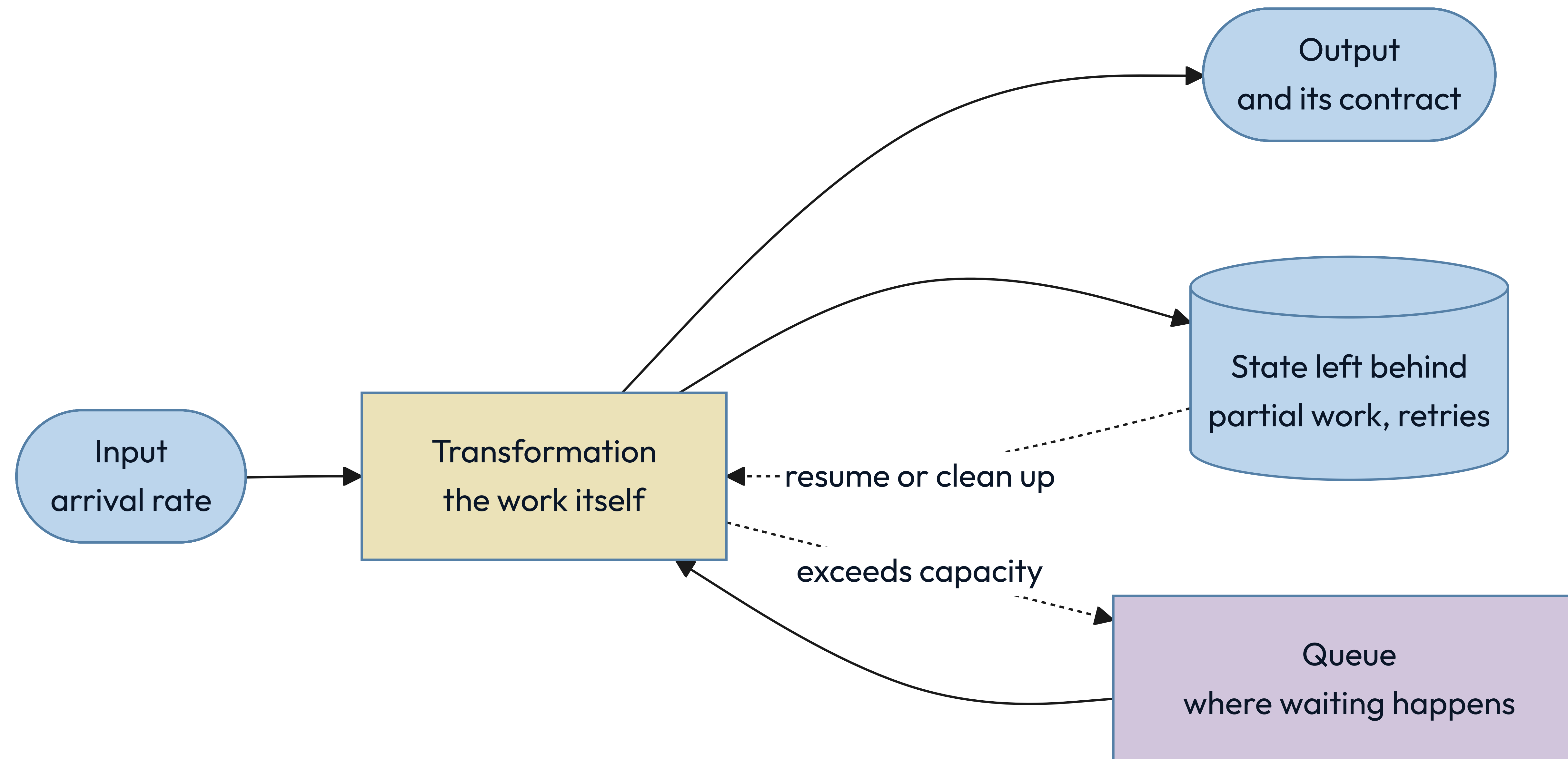
RATE IS PART OF THE DEFINITION

"Resize an image" is not a process until you say how many per second.

LEFTOVER STATE IS WHERE BUGS LIVE

Half-finished work, retries and duplicates all come from an interrupted run.

Every process is five things, and four of them are usually left unsaid.



A model is a simplification you chose on purpose.

What did I leave out, and can I defend it?

Not a smaller copy of the system. It keeps what matters for one question and drops the rest, so you reason faster than you build.

YOU KNOW YOU'RE HERE WHEN

You can name something real your drawing deliberately does not show.

FIDELITY IS NOT THE GOAL

A model as detailed as the system is as hard to think about.

EACH MODEL ANSWERS ONE QUESTION

The data-flow diagram is the wrong one for explaining failure.

TWO FACETS OF ONE IDEA

A model is useful because it is wrong in a way you control.

The value is not in what a model captures. It is in what it drops.

I**What it keeps**

The few variables that move the answer to the one question you are asking.

**II****What it drops**

Everything else, named out loud.
An unstated omission is a bug you have not found.

★ Ask an engineer what their diagram leaves out. The ones who can answer drew it on purpose.

A constraint is a limit that does not care what you prefer.

What is genuinely not negotiable?

Constraints narrow the design space, and that is the good news: a problem with no constraints has infinite answers and no way to choose.

YOU KNOW YOU'RE HERE WHEN

Naming the limit removes options rather than adding caveats.

REAL CONSTRAINTS ARE MEASURABLE

"Fast" is a wish. "Under 200ms at the 99th percentile" is a constraint.

SOME CONSTRAINTS ARE CHOSEN

A team of three and one cloud provider are elected limits you can revisit.

CONSTRAINT FAMILIES

Constraints come from four places, and they behave differently.

Physical

Light speed, disk seeks,
memory bandwidth.
Nothing negotiates
these.

Economic

Budget, headcount, the
price of a GPU-hour.
Money moves these
slowly.

Human

What the team knows,
what a user tolerates,
what a rotation
sustains.

Regulatory

Residency, retention,
consent, audit.
Breaking these beats
an outage.

KEY INSIGHT

Physical constraints set the ceiling. Human constraints decide whether you reach it.

An invariant is a sentence that must never become false.

What would mean this is broken, even with nothing crashed?

An invariant outranks a requirement. It holds during a deploy, a partition, a retry storm, and a live migration.

YOU KNOW YOU'RE HERE WHEN

The claim is true now and must still be true tomorrow.

WRITE THEM AS SENTENCES

"A balance is never negative." "Every order has exactly one payment."

THEY TELL YOU WHERE TO TEST

Each one names a check, an alert, and usually a database constraint.

Infrastructure is the part you rely on and do not want to think about.

What holds everything else up?

Compute, storage, network, identity, deployment, observability. The word names a relationship, not a technology: infrastructure is whatever you build on and expect to keep working.

THE TEST

If it vanishing stops many unrelated things at once, it is infrastructure.

IT IS SOMEONE ELSE'S PRODUCT

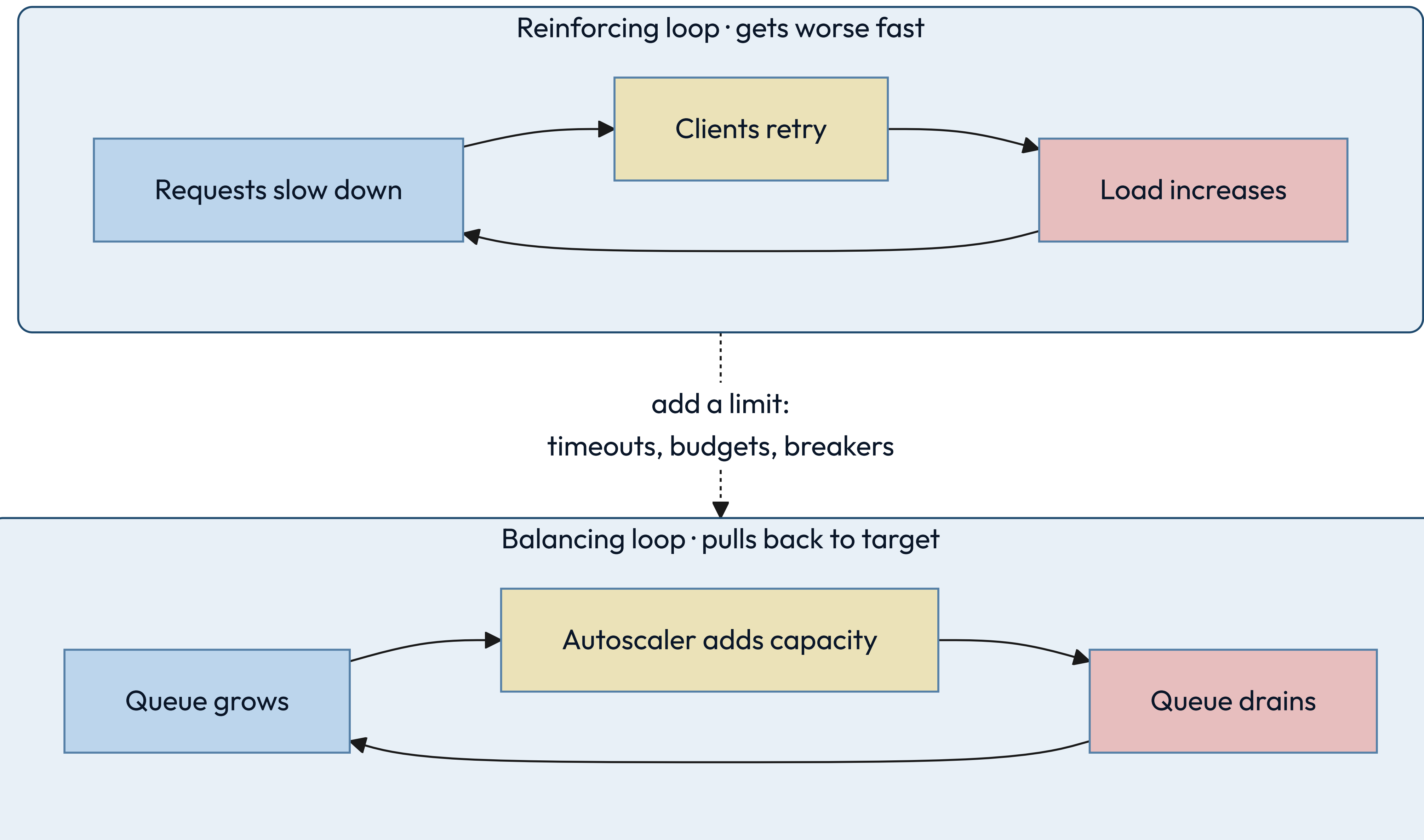
Your infrastructure is another team's system, with its own invariants.

BORING IS THE REQUIREMENT

It earns its place by being predictable, not by being interesting.

HOW SYSTEMS MISBEHAVE

Feedback loops explain almost every surprise a system gives you.



Three questions turn a vague ask into a bounded problem.

STEP 01

Name the boundary

What is inside, what is outside, who owns each side.



STEP 02

Name the environment

What arrives uninvited: load, failures, adversaries, rules.



STEP 03

Name the constraints

Physical, economic, human, regulatory — as numbers where numbers exist.

Three more turn a bounded problem into a design.

STEP 01

Name the invariants

The sentences that must never go false, and how you would notice.



STEP 02

Choose the solution type

An MVP and an optimal build are different answers to the same words.



STEP 03

Find the bottleneck

One resource limits the whole; everything else is decoration until it moves.

02

PART TWO

**Model a Tuesday before
you model a datacenter.**

THE WORKED EXAMPLE

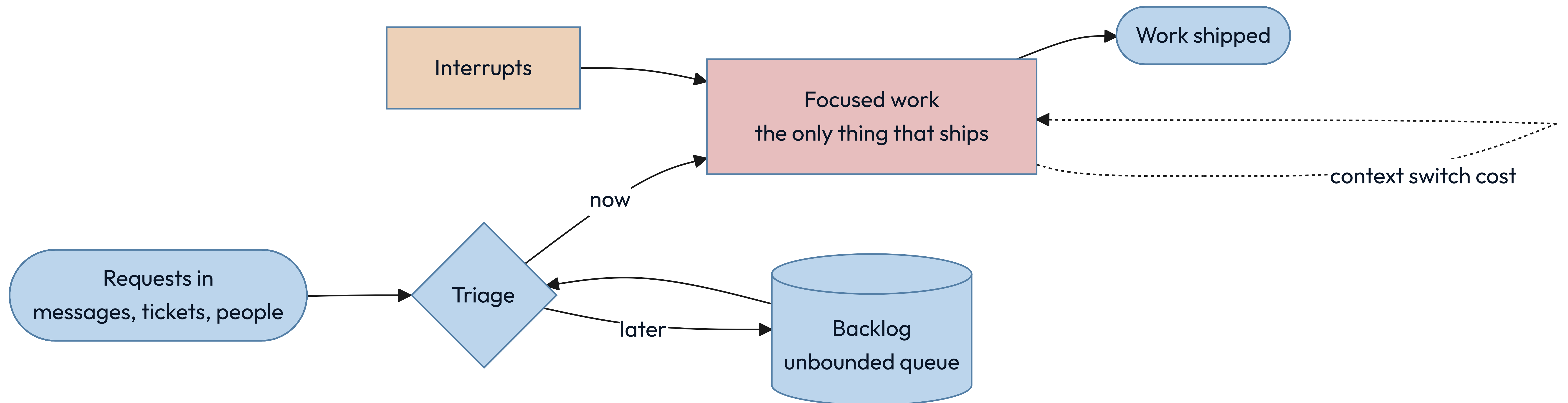
You already run a system with hard constraints and a real bottleneck.

Your day has inputs you do not choose, a boundary you defend badly, constraints you never wrote down, and at least one queue that keeps growing. Modeling it takes ten minutes and teaches every move you will use on a distributed system.

We will do it once, in full, and then name what transfers.

STEP ONE • DRAW THE PROCESS

A day is a pipeline with one shared resource and no admission control.



STEP TWO • NAME THE PARTS

Four questions describe a day exactly as they describe a datacenter.

- 01

Elements

Your attention, your calendar, your inbox, your colleagues, your tools.
- 02

Boundary

What you can decline is inside. What you must answer is outside.
- 03

Environment

Meetings you did not book, outages, a manager's priorities, your own energy curve.
- 04

Purpose

What the day is actually for, stated as one outcome rather than a task list.

STEP THREE • NAME THE CONSTRAINTS

Four limits shape every day, and only one of them is time.

Attention

Roughly four hours of deep work exist per day. Physical, not negotiable.

Switching

Every interrupt costs the reload, not just the interruption.

Dependency

Work blocked on a review moves at someone else's pace.

Energy

Capacity is not flat.
The same task costs
more at 5pm than at
9am.

KEY INSIGHT

Only one of these is measured in hours, which is why a calendar never fixes a day.

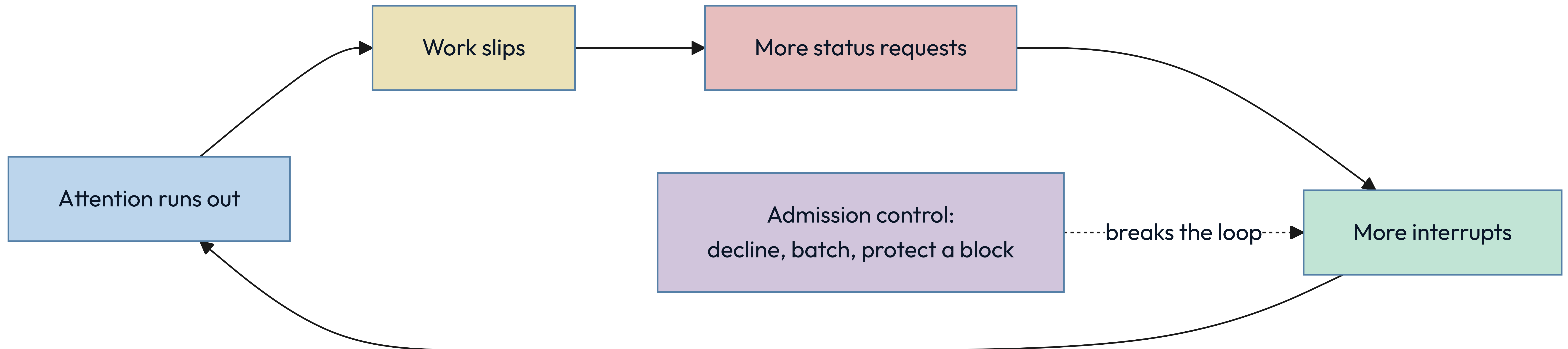
STEP FOUR • THE BOTTLENECK

4h

of deep work per day, versus the 8 hours
a calendar sells you. Everything else
queues behind it.

STEP FIVE • FIND THE LOOP

Bad days repeat because the system is wired to reinforce them.



STEP SIX • STATE THE INVARIANT

An invariant on your day reads exactly like one on a service.

What must never go false?

"One protected block of two hours exists every working day." It survives a busy week, a launch, and a manager's reorganized calendar — or it was never an invariant, only a preference.

YOU KNOW YOU'RE HERE WHEN

You can name the alert: the day the block disappears, something is wrong.

IT CONSTRAINS OTHER PEOPLE

A real invariant changes what others may put on your calendar.

IT IS FALSIFIABLE

You can look at last week and say plainly whether it held.

THE TRANSFERABLE METHOD

Five moves model anything, from a Tuesday to a global service.

STEP 01

Draw the process

Inputs, transformation, outputs, and every queue between them.

STEP 02

Name the boundary

Split what you control from what you only negotiate with.

STEP 03

Measure the constraints

Write each limit as a number, even a rough one.

STEP 04

Find the bottleneck and the loop

One resource is scarce; one feedback loop makes it worse.

Every move you just made has a direct counterpart in software.

The backlog is an unbounded queue. Interrupts are context switches. Declining work is admission control. A protected block is a reserved resource pool. The energy curve is a system that performs differently under sustained load than in a benchmark.

Nothing about the method changes when the parts become machines. Only the units do.

03

PART THREE

"Design Instagram" is
five different questions.

Premature optimization is the root of all evil. Yet we should not pass up our opportunities in that critical three percent.

DONALD KNUTH, 1974 — USUALLY QUOTED WITH THE SECOND SENTENCE REMOVED

THE FRAMING QUESTION

Before you design anything, ask which kind of answer is wanted.

The same words — "design a photo sharing app" — have five legitimate answers that share almost no architecture. An MVP that takes a week and a specialized build that takes two years are both correct, for different questions.

Getting this wrong is the most expensive mistake in the room, and it happens before a single box is drawn.

Five kinds of solution, and each one is right somewhere.

Each rung costs more, commits harder, and buys a different thing. You move up only when the rung below stops paying.

01	MVP	Buys information fast.	Does anyone want this?
02	Scaled	Holds up as load grows.	Will it survive success?
03	Optimized	Cuts cost on a known path.	Where is the money going?
04	Optimal	Provably best, for one model.	What is the real limit?
05	Specialized	An advantage nobody can copy.	What can only we do?

An MVP is an experiment wearing a product's clothes.

Does anyone want this?

Its job is to produce a decision, not to last. Every hour spent making it durable is an hour spent on a system you may correctly delete next month.

YOU KNOW YOU'RE HERE WHEN

The riskiest thing is demand, not scale, and a wrong answer is cheap.

BUY SIMPLICITY, NOT CAPACITY

One database, one service, one region, boring technology, no cleverness.

THE EXIT IS PLANNED

Name in advance the metric that means "stop, this must now be built properly."

A scaled system survives ten times the load without a rewrite.

Will it survive success?

You are no longer buying information; you are buying headroom. The design question shifts from "does it work" to "what breaks first, and how do I move that limit."

YOU KNOW YOU'RE HERE WHEN

Growth is real and the current design has a limit you can name.

ADD AXES, NOT MACHINES

Statelessness, partitioning, caching and queues buy scale. A bigger box only postpones.

COST PER UNIT MATTERS NOW

Cost per request stops being noise and starts being the second constraint.

Optimizing means moving one measured number on one known path.

Where is the cost actually going?

Optimization without a profile is decoration. You need the measurement first, the target second, and the willingness to accept the complexity you are about to add.

YOU KNOW YOU'RE HERE WHEN

A profile shows which tenth of the work is most of the cost.

THE PATH MUST BE HOT

Optimizing a path that runs twice a day avoids the real problem.

COMPLEXITY IS THE PRICE

Each optimization narrows the assumptions the system may safely break.

Optimal means provably best against an objective you wrote down.

What is the actual limit?

This is a rarer thing than it sounds. It requires a stated objective, a stated model, and a proof or a bound — and it is only optimal for the assumptions you fixed.

YOU KNOW YOU'RE HERE WHEN

The objective is written as a function and the constraints are written as inequalities.

THE PROOF IS AGAINST A MODEL

Change the assumptions and the optimal answer changes with them.

IT AGES BADLY

An optimal design pinned to last year's hardware is a legacy system with a certificate.

A specialized system trades generality for an advantage nobody can copy.

What can only we do?

Custom silicon, a purpose-built storage engine, a physics-aware scheduler. You give up flexibility, portability and hiring pool in exchange for a capability the market cannot buy.

THE SIGNAL

The advantage is durable, measurable, and central to why customers choose you.

THE COST IS PERMANENT

You now maintain what everyone else gets for free from a vendor.

ALMOST NOBODY IS HERE

Most teams reaching for this rung actually needed the optimized one.

THE LADDER, COMPARED

Each rung buys a different thing, and charges for it differently.

TYPE	BUYS	COSTS	REVERSIBLE
MVP	Information	Almost nothing	Fully
Scaled	Headroom	Design effort	Mostly
Optimized	Cost or latency	Complexity	Partly
Optimal	A proven bound	Rigor and assumptions	Rarely
Specialized	An advantage	Permanent maintenance	No

✦ *Move up one rung at a time, and only when the rung you are on has stopped paying.*

WHEN TO CLIMB

Two questions decide which rung you belong on.

Low uncertainty · Low cost of error.

Ship the MVP

Learn from real usage

Low uncertainty · High cost of error.

Optimize or prove optimal

The path is known and expensive

High uncertainty · Low cost of error.

Always the MVP

Buy information before capacity

High uncertainty · High cost of error.

Reduce uncertainty first

Prototype, model, measure, then build

THE RULE

Climb the ladder one rung at a time, and never skip on a hunch.

MOVE UP WHEN THE CURRENT RUNG
FAILS ON EVIDENCE

A measured limit, a real cost, a
named risk — not an anticipated
one.

MOVE UP ONE RUNG ONLY

Skipping from MVP to optimal
buys rigor for assumptions you
have not yet tested.

MOVE BACK DOWN WHEN THE EVIDENCE
CHANGES

A rewrite that simplifies is a
legitimate move, not an
admission of failure.

04

PART FOUR

Six kits, to keep for
the rest of your career.

HOW TO READ A KIT

Every kit entry answers the same three questions, on purpose.

Reach for it when. Walk away when. The constraint you inherit. One shape for every entry, so you can compare two options without re-reading a manual. Each kit then closes with the invariants that hold across all of its entries.

The entries name concepts, not products. Products change every three years; the constraint an append-only store puts on your reads does not.

KIT ONE

**Data: where state lives,
and what that costs you.**

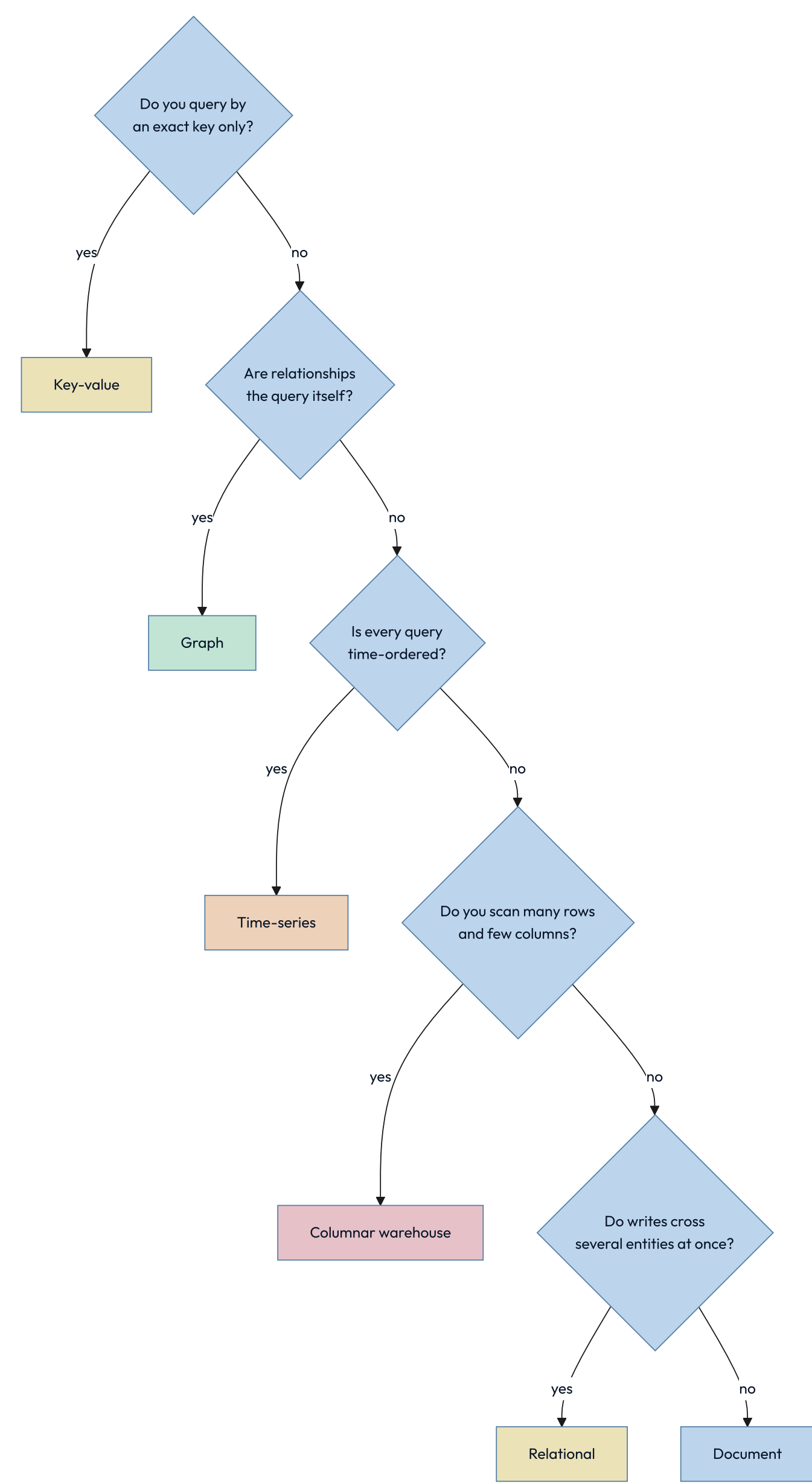
Five questions pick a data store, and the product name comes last.

Answer these before naming a technology. Most arguments about databases are really disagreements about question two.

01	Shape	How the data is structured.	Rows, documents, or edges?
02	Access	How it is read and written.	Lookups, ranges, or scans?
03	Consistency	What a reader may see.	Must it be the latest?
04	Scale	Size, throughput, growth.	Does it fit one machine?
05	Cost	Money, operations, skill.	Who runs it at 3am?

THE DECISION TREE

Access pattern picks the store; the data's shape only narrows the field.



A relational store is the default until you can name why it is not.

Reach for it when

Entities relate, writes touch several at once, and the questions will change.

Walk away when

One table outgrows one machine's writes, or the schema genuinely differs per row.

The constraint you inherit

Joins and transactions need a coordinator. Self-sharding loses both; a distributed engine charges a round trip.

A key-value store is a hash map with an operations team.

Reach for it when

You always know the exact key, and you need sub-millisecond rather than tens of milliseconds.

Walk away when

You need to ask any question about the value's contents.

The constraint you inherit

No secondary access path. Every new query means a new key you must write yourself.

A document store trades joins for keeping one object whole.

Reach for it when

Reads fetch one self-contained object and the shape varies between records.

Walk away when

The same fact appears in many documents and must stay consistent across them.

The constraint you inherit

Denormalized copies. Every update is a fan-out you now own.

A wide-column store buys enormous write throughput with a fixed query shape.

Reach for it when

Writes are relentless, and every read is a partition key plus a sorted range.

Walk away when

Query patterns are still moving, or you need cross-partition transactions.

The constraint you inherit

The primary key is the schema. Changing how you query means rewriting the data.

A graph store is for questions that traverse, not questions that filter.

Reach for it when

The query walks relationships of unknown depth: reachability, paths, recommendations.

Walk away when

You have relationships but only ever join two hops. A relational store does that faster.

The constraint you inherit

Traversals are hard to partition, so scale-out is genuinely harder here than elsewhere.

Some stores hold truth, some borrow it, and the split surprises people.

The five above can be where a fact lives — and so can two below: an object store is usually where media first lands, and a time-series store is where a metric first lands. Nothing regenerates either.

The genuinely derived ones are search, vector, cache and warehouse. Anything derived must be rebuildable, must be allowed to lag, and must never be the only copy.

A time-series store assumes time is the primary key and never changes.

Reach for it when

Appends dominate, records are immutable, and every read has a time range.

Walk away when

Records get updated, or the interesting axis is anything other than time.

The constraint you inherit

Retention and downsampling become design decisions, not settings.

A search index is a derived view, never a source of truth.

Reach for it when

Users query text they typed, and ranking and relevance matter more than exactness.

Walk away when

The result must be transactionally correct at the instant it is read.

The constraint you inherit

It lags. Every search index is eventually consistent with whatever feeds it.

A vector index answers "most similar to this", not "equal to this".

Reach for it when

Retrieval is by meaning: semantic search, recommendations, grounding a model's answer.

Walk away when

An exact filter or a keyword match would do. It is slower and approximate.

The constraint you inherit

Results are approximate by construction, and the embedding model becomes part of your schema.

An object store is the cheapest durable place to put large immutable bytes.

Reach for it when

Items are large, written once, read by key, and must survive for years.

Walk away when

You need to modify part of an object, or list and filter by content.

The constraint you inherit

Listing is slow and expensive. The index of what you stored belongs somewhere else.

A cache is a bet that the same answer will be wanted again soon.

Reach for it when

Reads repeat, the source is expensive, and slightly stale is acceptable.

Walk away when

Staleness is unsafe, or the working set is larger than the cache.

The constraint you inherit

Invalidation. A cache converts a correctness problem into a timing problem.

A durable log turns "do it now" into "do it reliably, soon."

Reach for it when

Producers outpace consumers, or several systems need the same events.

Walk away when

The caller needs the result in the same request.

The constraint you inherit

At-least-once delivery. Every consumer must be idempotent or you will double-charge someone.

A warehouse is built to scan billions of rows and read four columns.

Reach for it when

Questions are aggregate, analysts write them, and minutes are an acceptable answer time.

Walk away when

A user is waiting for the result, or you need single-row updates.

The constraint you inherit

It is loaded from somewhere else, so it is always behind the operational store.

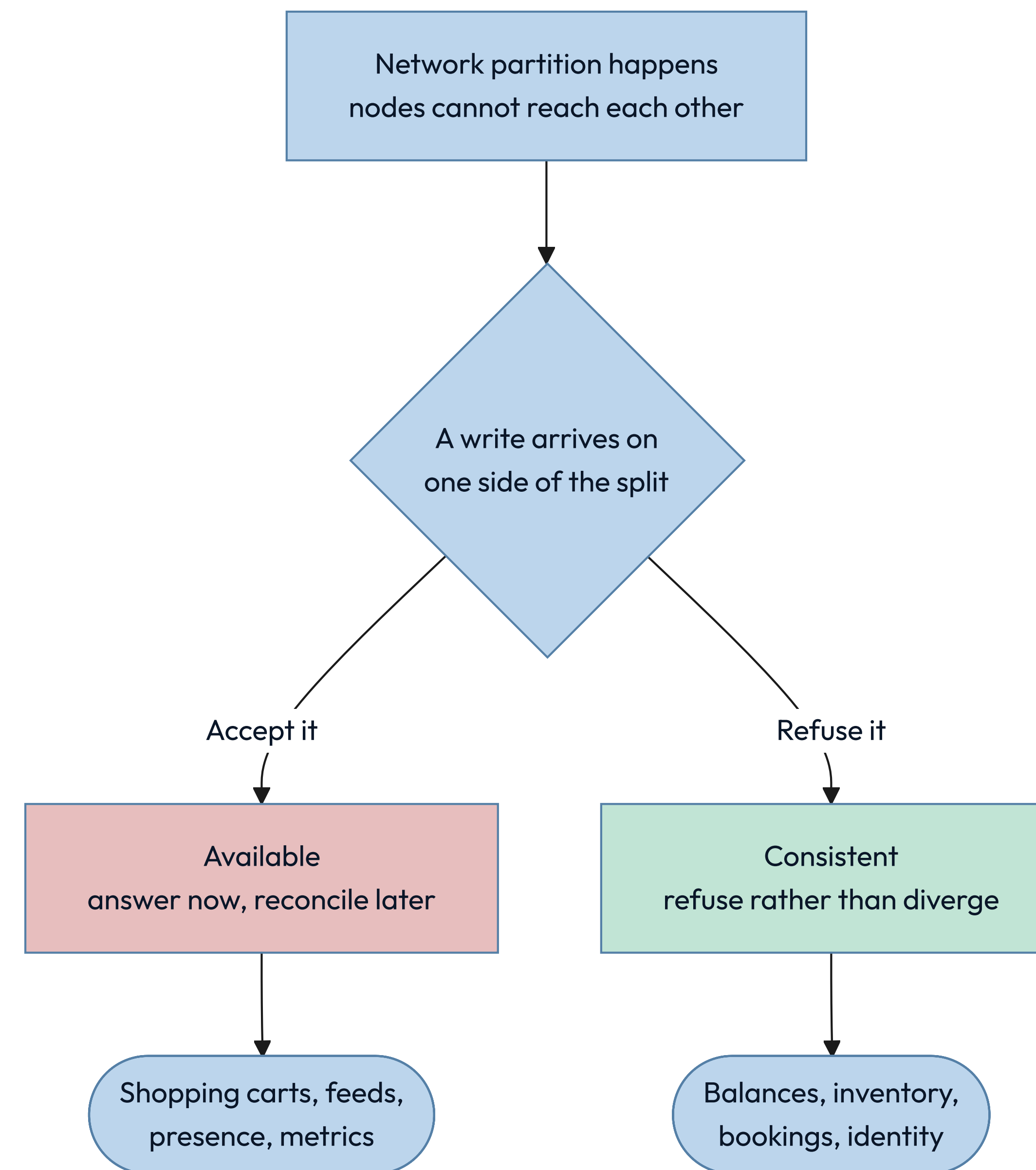
THE SCAN TABLE

Five columns compare any two stores you are choosing between.

STORE	ACCESS	CONSISTENCY	SCALES BY	WEAK AT
Relational	Key, range, join	Strong	Replicas, then partitioning	Cross-shard writes
Key-value	Exact key	Tunable	Horizontal	Any other query
Document	Key, secondary index	Per document	Horizontal	Cross-document facts
Wide-column	Partition plus range	Tunable	Horizontal	New query patterns
Object store	Key	Read-after-write	Effectively unbounded	Listing, partial updates

THE THEOREM EVERYONE MISQUOTES

CAP is a choice you make during a partition, and only then.



THE EVERYDAY VERSION

PACELC is the more useful theorem, because partitions are rare and latency is not.

DURING A PARTITION

Choose availability or consistency. This is CAP, and it applies for the few minutes a year a link is actually broken.



IN NORMAL OPERATION

Choose latency or consistency. Every replicated write pays a round trip to be safe, and that is the choice you make a billion times a day.

THE CONSISTENCY LADDER

Consistency is a ladder, not a switch, and each rung has a price.

01	Linearizable	The latest write, always. Costs a round trip.
02	Sequential	One shared order, possibly behind. Cheaper.
03	Causal	Related events ordered, unrelated ones maybe not.
04	Read your writes	You see your own edits. The least a live editor accepts.
05	Eventual	Replicas converge in the end. Cheapest.

TWO TRANSACTION PHILOSOPHIES

ACID and BASE are not rivals; they are answers to different questions.

Both are correct. The question is whether the cost of being briefly wrong is higher than the cost of coordinating.

I ACID

Coordinate first, so the data is never observably wrong. Pay in latency and in a limit on how far you can partition.



II BASE

Accept divergence, converge afterwards. Pay in application complexity: every reader must tolerate stale or conflicting state.

An index is a second copy of your data, sorted for one question.

What does it really cost?

Indexes make reads fast by making writes slower and storage larger. Every index is a promise to maintain a sorted structure on every single write, forever.

YOU KNOW YOU'RE HERE WHEN

You can name the exact query each index exists to serve.

THE COST IS ON WRITES

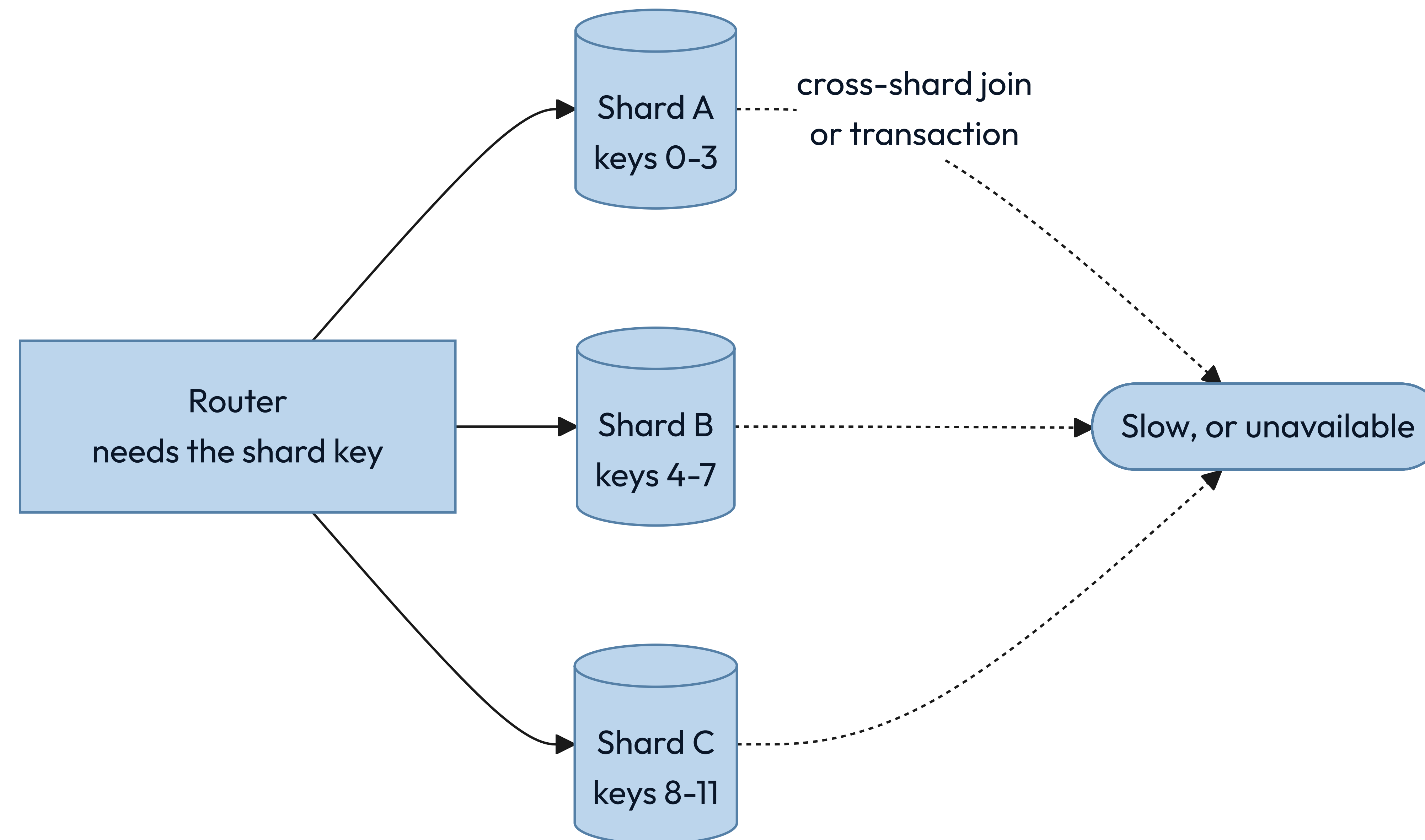
Five indexes mean five extra structures updated per insert.

SELECTIVITY DECIDES, NOT CARDINALITY

Three evenly spread values get ignored; three where one is 0.1 percent do not.

PARTITIONING

Sharding buys throughput by giving up the questions that cross shards.



CHOOSING A SHARD KEY

The shard key is the hardest decision, and the one you cannot undo cheaply.

By hash of the key

Spreads load evenly, destroys range queries. The safe default.

By range

Keeps ranges fast, invites hot spots on the newest range.

By tenant or region

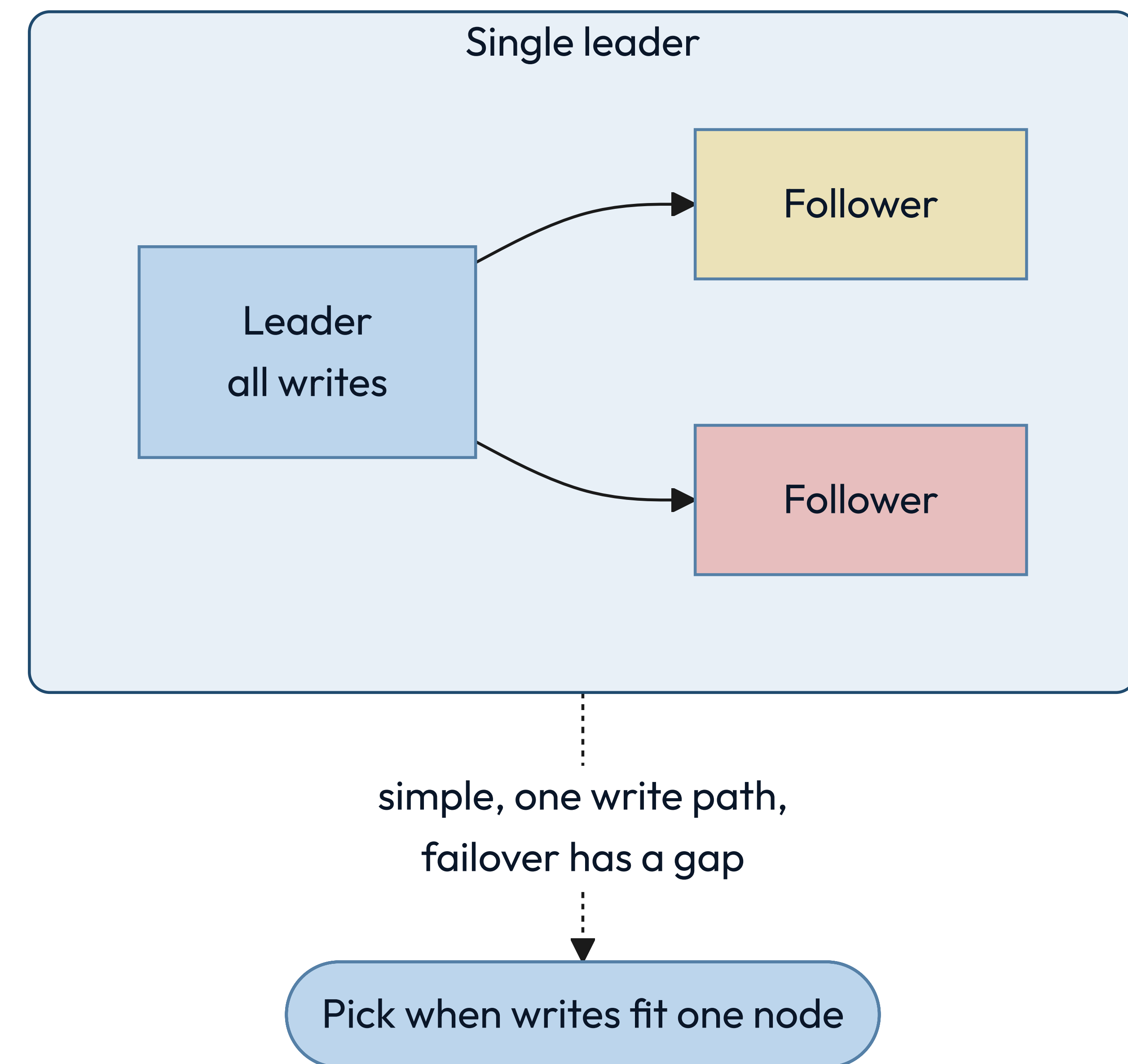
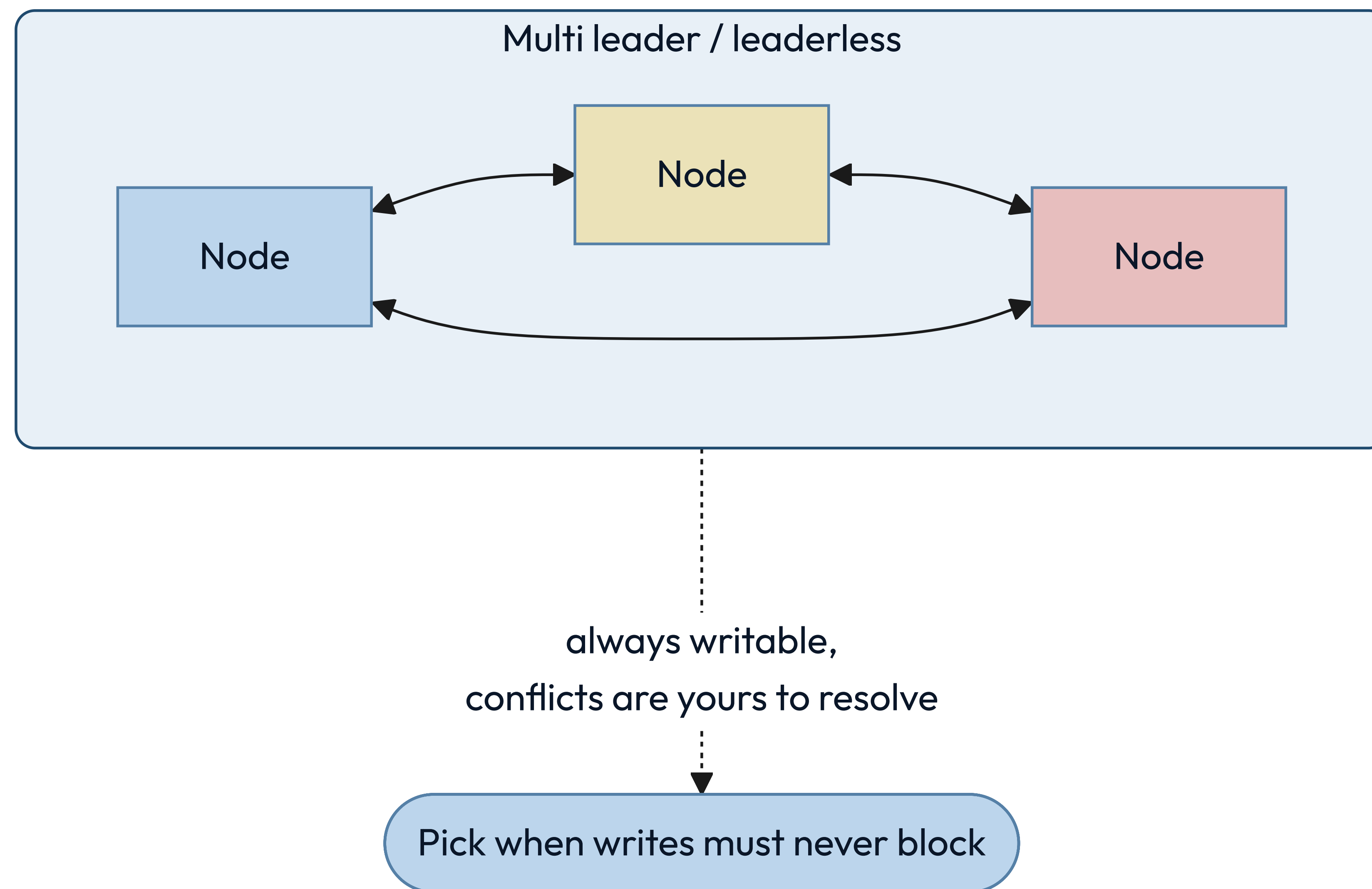
Matches the access pattern and the law, until one tenant grows enormous.

KEY INSIGHT

A skewed shard key produces a system where one machine is on fire and the rest idle.

REPLICATION

Replication topology decides what you lose when a machine dies.



ISOLATION

Isolation levels name exactly which anomaly you have agreed to tolerate.

LEVEL	PREVENTS	STILL ALLOWS	TYPICAL USE
Read committed	Dirty reads	Non-repeatable reads	The common default
Repeatable read	Non-repeatable reads	Phantoms, write skew	Reports inside a transaction
Snapshot	Most read anomalies	Write skew	Read-heavy applications
Serializable	Every ordering anomaly	Real-time ordering, unless strict	Money, inventory, bookings

✦ *Read committed is the default in most systems, which means most applications tolerate anomalies they never chose.*

Four sentences should hold in any data design you ship.

01

Exactly one source of truth per fact

Every other copy is derived, and is labeled as derived.

02

Every derived store can be rebuilt

If the search index burns down, a job restores it from the source.

03

Every consumer of a queue is idempotent

At-least-once delivery is the only delivery you actually get.

04

Every write path has a stated consistency level

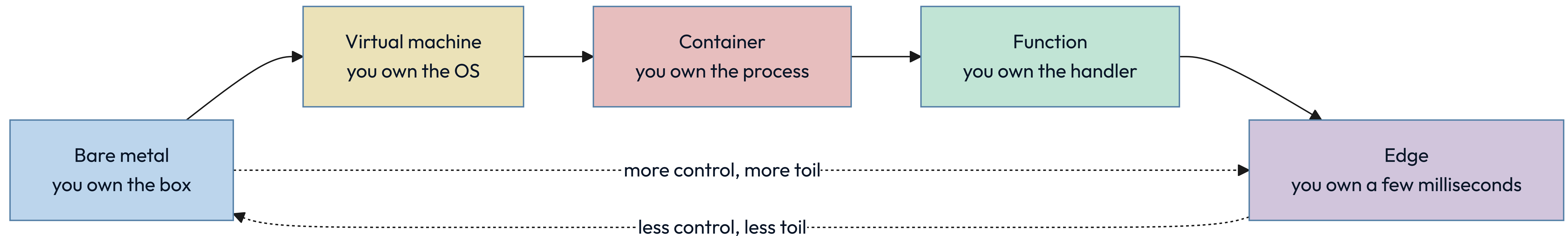
"Whatever the database does by default" is not a stated level.

KIT TWO

**Compute: where work happens,
and who owns the machine.**

THE COMPUTE LADDER

Every rung hands more of the machine to someone else, and takes flexibility back.



A virtual machine is the unit you reach for when the process is long-lived.

Reach for it when

The workload runs continuously, holds state in memory, or needs unusual kernel settings.

Walk away when

Traffic is spiky and idle machines dominate the bill.

The constraint you inherit

You now own patching, capacity planning, and the difference between staging and production.

A container makes the deployable unit identical everywhere it runs.

Reach for it when

You run many services, deploy often, and want one packaging story across all of them.

Walk away when

You have one small service. An orchestrator costs more than it saves at that size.

The constraint you inherit

The orchestrator becomes infrastructure, with its own failure modes and its own on-call.

A function is compute you rent by the millisecond, so idle costs you nothing.

Reach for it when

Traffic is spiky or rare, work is short, and per-request isolation is welcome.

Walk away when

Runs are long, or steady traffic makes per-request pricing more expensive than a machine.

The constraint you inherit

Cold starts unless you pay to keep instances warm, which is paying for idle again.

Edge compute buys distance, and pays for it in capability.

Reach for it when

The work is small, the user is far away, and the round trip is the cost.

Walk away when

The work needs your database, which is not at the edge.

The constraint you inherit

Tight memory and time limits, a restricted runtime, and data that is far from the code.

THE PROPERTY THAT DECIDES EVERYTHING

Statelessness is not a style; it is what makes a machine replaceable.

STATEFUL SERVICES

The instance holds something no other instance has: a session, a lock, a cache, a connection. Scaling means moving state, and failure means losing it.



STATELESS SERVICES

Any instance can serve any request because the state lives elsewhere. Scaling is arithmetic, and failure is a routing change.

TWO WAYS TO PROCESS

Batch and stream are the same computation with different latency budgets.

BATCH

Collect, then process a bounded set on a schedule.
Simple to reason about, easy to re-run, and always behind by one interval.

**STREAM**

Process each event as it arrives, holding windows and watermarks. Fresh answers, at the cost of handling late and out-of-order events forever.

Anything you run should satisfy these four claims.

01

Any instance can be killed without a customer noticing

If that is false, you have hidden state you have not named.

02

Deployments are reversible

A rollback is a routine operation, not an incident response.

03

Capacity is a number someone owns

Not "it autoscales" — the ceiling, the cost, and who is paged when it is hit.

04

Startup does not depend on startup order

Services retry into each other rather than requiring a sequence.

KIT THREE

**Network: distance is a cost
you cannot optimize away.**

THE NUMBERS TO MEMORIZE

Five numbers explain most latency arguments before they start.

01	L1 cache read	1 ns
02	Main memory read	100 ns
03	Flash random read	0.1 ms
04	Datacenter hop	0.5 ms
05	Cross-continent hop, measured	150 ms

WHY THE NUMBERS MATTER

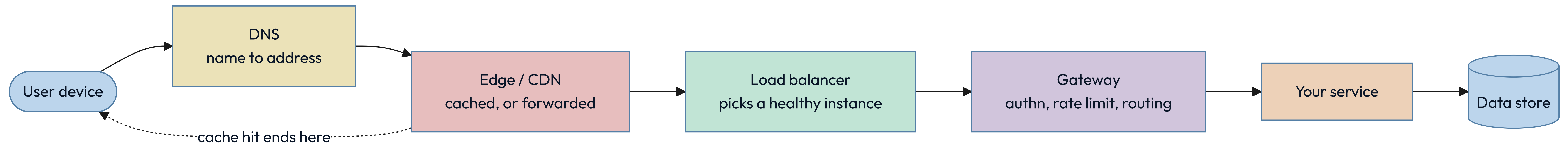
Light in fiber travels about 200 kilometers per millisecond, and that is final.

The measured 150 milliseconds is well above that fiber floor, because packets do not travel in straight lines. A design needing three sequential intercontinental round trips spends half a second before it executes an instruction.

This is why replication, caching and edge delivery exist. All three are ways of buying distance, and none of them makes distance free.

THE REQUEST PATH

Four hops sit between a tap and your service, and each one can fail.



DNS is the first request, the slowest to change, and a real dependency.

Reach for it when

You need a stable name in front of an address that will change.

Walk away when

You need failover in seconds. Caches downstream ignore your intentions.

The constraint you inherit

TTL is a promise about the past. Old answers stay in resolvers you cannot reach.

A CDN moves bytes closer to readers and moves nothing else.

Reach for it when

Content is large, popular, and the same for many users.

Walk away when

Every response is personalized and cacheable for nobody but one person.

The constraint you inherit

Purging is eventual. Versioned URLs beat invalidation every time.

TWO WAYS TO BALANCE LOAD

A layer-four balancer moves packets; a layer-seven balancer reads requests.

LAYER FOUR

Routes by address and port without looking inside. Very fast, protocol agnostic, and blind to what the request actually asks for.

**LAYER SEVEN**

Parses the request, so it can route by path, retry safely, split traffic and terminate encryption. More capable, more expensive, and now it is a system with opinions.

Four API styles, and the one question that separates them.

STYLE	BEST AT	WEAK AT	CHOOSE WHEN
REST	Cacheable resources	Chatty multi-entity reads	Public, resource-shaped APIs
RPC	Typed internal calls	Browser reach, client coupling	Service-to-service traffic
GraphQL	Client-shaped queries	Caching, cost control	Many clients, one backend
Streaming	Continuous updates	Simplicity, statelessness	Live feeds and progress

A request without a timeout is a resource leak waiting for a bad day.

What does an unbounded wait actually cost?

Every waiting request holds a connection, a thread and memory. Under a slow dependency, unbounded waits convert one slow service into a total outage.

YOU KNOW YOU'RE HERE WHEN

Every outbound call has a deadline, and the deadline shrinks as it propagates.

RETRIES NEED A BUDGET

Retrying without a cap turns a brief failure into a self-inflicted flood.

BACKOFF MUST BE RANDOM

Synchronized retries arrive together and rebuild the exact spike you just survived.

A call that leaves your process owes you four promises.

01

Every remote call has a deadline

Inherited from the caller, and always shorter than the caller's own.

02

Every retry has a budget and jitter

Bounded attempts, randomized delays, and a breaker when the target is down.

03

Every write is idempotent or keyed

The network will deliver your request twice. Decide now what that means.

04

Distance appears in the design

Round trips between regions are counted, not discovered in production.

KIT FOUR

Scale: four moves, and knowing which one the bottleneck needs.

There are only four ways to make a system handle more.

Everything else is a variation.

When someone proposes a scaling change, ask which of these four it is, and which bottleneck it moves.

01	Reduce	Do less work per request.	Can this be reused?
02	Spread	Split work across machines.	Can this be partitioned?
03	Defer	Move work out of the request.	Must this happen now?
04	Duplicate	Copy data closer to readers.	Are reads the pressure?

THE ONE FORMULA TO MEMORIZE

Concurrency, throughput and latency are one equation, not three numbers.

$$L = \lambda W$$

WHERE

- L — requests in the system at once (concurrency)
- λ — arrival rate (requests per second)
- W — time each request spends inside (latency)

USING IT

The formula tells you the thread pool size before you guess it.

At 2,000 requests per second with an average of 50 milliseconds each, 100 requests are in flight at any moment. That is your minimum concurrency, and no amount of tuning makes it smaller while both other numbers hold.

It also explains the failure: if latency triples during an incident, concurrency triples too, and a pool sized for the good case is now the outage.

The average request is a fiction; users live in the tail.

Why does a fast service feel slow?

A page that makes 100 parallel calls waits for the slowest. With a one-percent chance of a slow call, 63 percent of pages hit at least one.

YOU KNOW YOU'RE HERE WHEN

You report the 99th percentile, not the mean, and you report it per dependency.

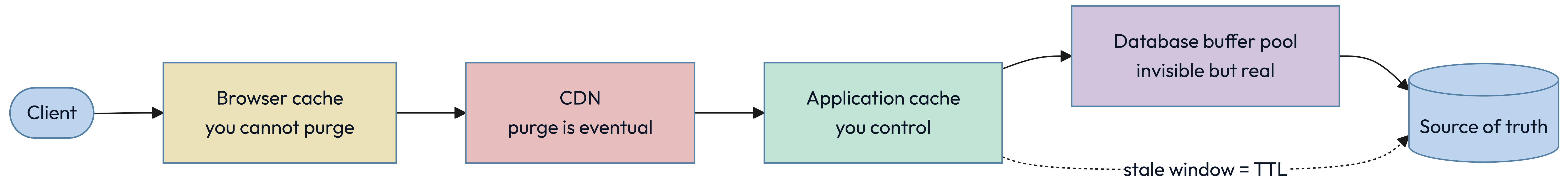
FAN-OUT AMPLIFIES IT

More parallel calls make a rare slow response into a common slow page.

HEDGING BUYS IT BACK

Send a duplicate request after the 95th percentile and take whichever answers first.

A cache exists at every layer, and each one lies to you differently.



CACHE STRATEGIES

Four caching patterns, and the failure each one owns.

Cache-aside

The app fills the cache on a miss. Simple, and stampedes on a cold key.

Read-through

The cache fetches for you. Cleaner code, and the cache is now on the critical path.

Write-through

Write both together. Always fresh, and every write pays the cache's latency.

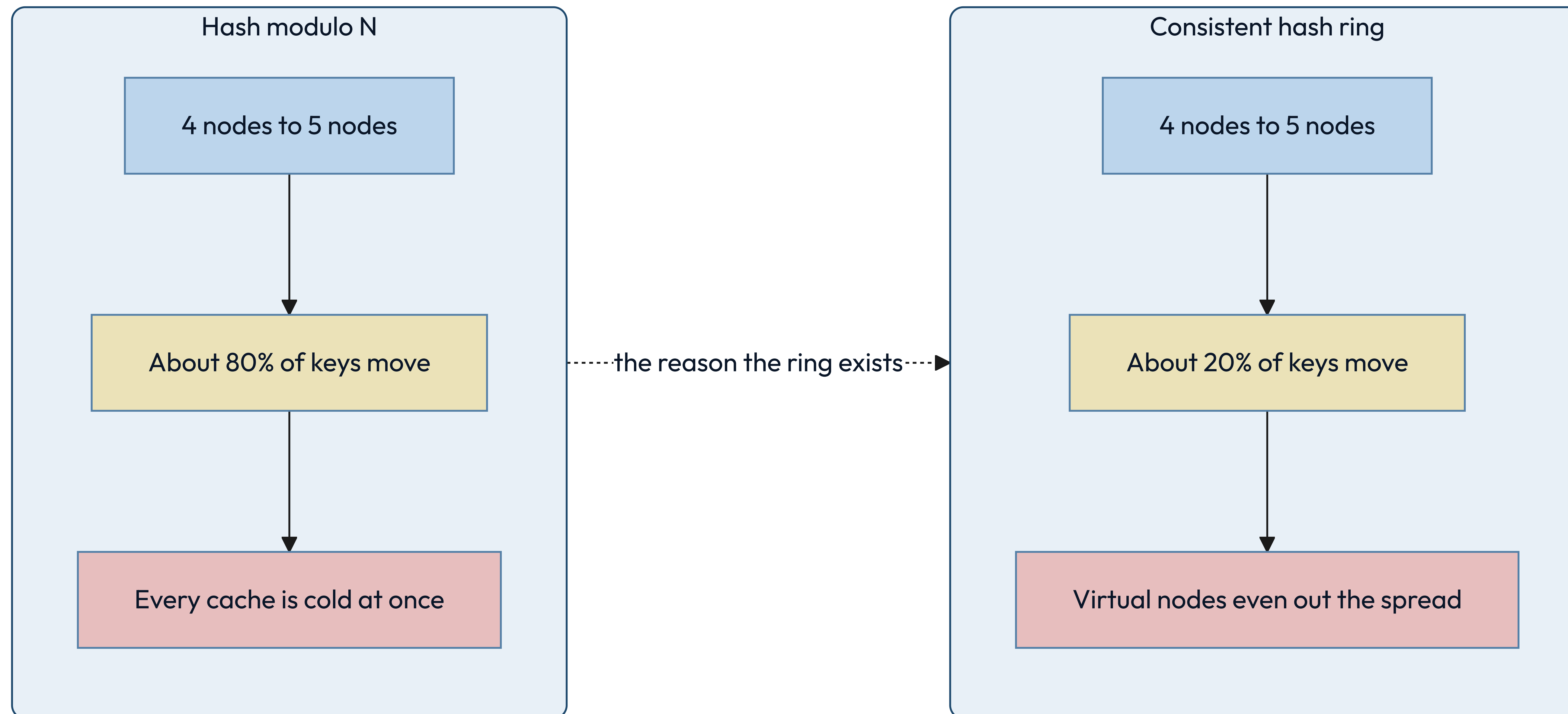
Write-behind

Write the cache, flush later. Fastest writes, and a crash loses them.

KEY INSIGHT

Choose by which failure you can survive, not by which pattern reads best in code.

Consistent hashing exists so adding a machine does not move all the keys.



Idempotency is what makes a retry safe, and retries are not optional.

What happens when the same request arrives twice?

Networks duplicate. Clients retry. Queues redeliver. The only question is whether the second delivery is harmless or charges a customer again.

YOU KNOW YOU'RE HERE WHEN

Every mutating endpoint takes a client-supplied key and deduplicates on it.

THE KEY COMES FROM THE CALLER

Generated before the first attempt, reused on every retry of that same intent.

STORE THE RESULT, NOT JUST THE FACT

A repeat must return the original answer, not an error saying it already happened.

A rate limit protects the system from its users, and from itself.

Reach for it when

Any shared resource can be exhausted by one caller, deliberately or accidentally.

Walk away when

The limit is applied so late that the work is already done when you reject.

The constraint you inherit

Limits are per-key state, so a distributed limiter is itself a consistency problem.

THE OLDEST QUESTION

Scaling up is faster to do; only scaling out keeps working.

VERTICAL, A BIGGER MACHINE

No code changes, no distribution, no new failure modes. It works until the largest machine is not large enough, and it never removes the single point of failure.

**HORIZONTAL, MORE MACHINES**

Unbounded headroom and real redundancy, bought with partitioning, coordination and a distributed system's whole class of new bugs.

Check these four before calling any design scalable.

01

The bottleneck is named and measured

Not suspected. A number, a graph, and the resource it belongs to.

02

Every queue is bounded

An unbounded queue converts a throughput problem into a memory outage.

03

Load sheds before it collapses

The system rejects work at the edge rather than failing everywhere at once.

04

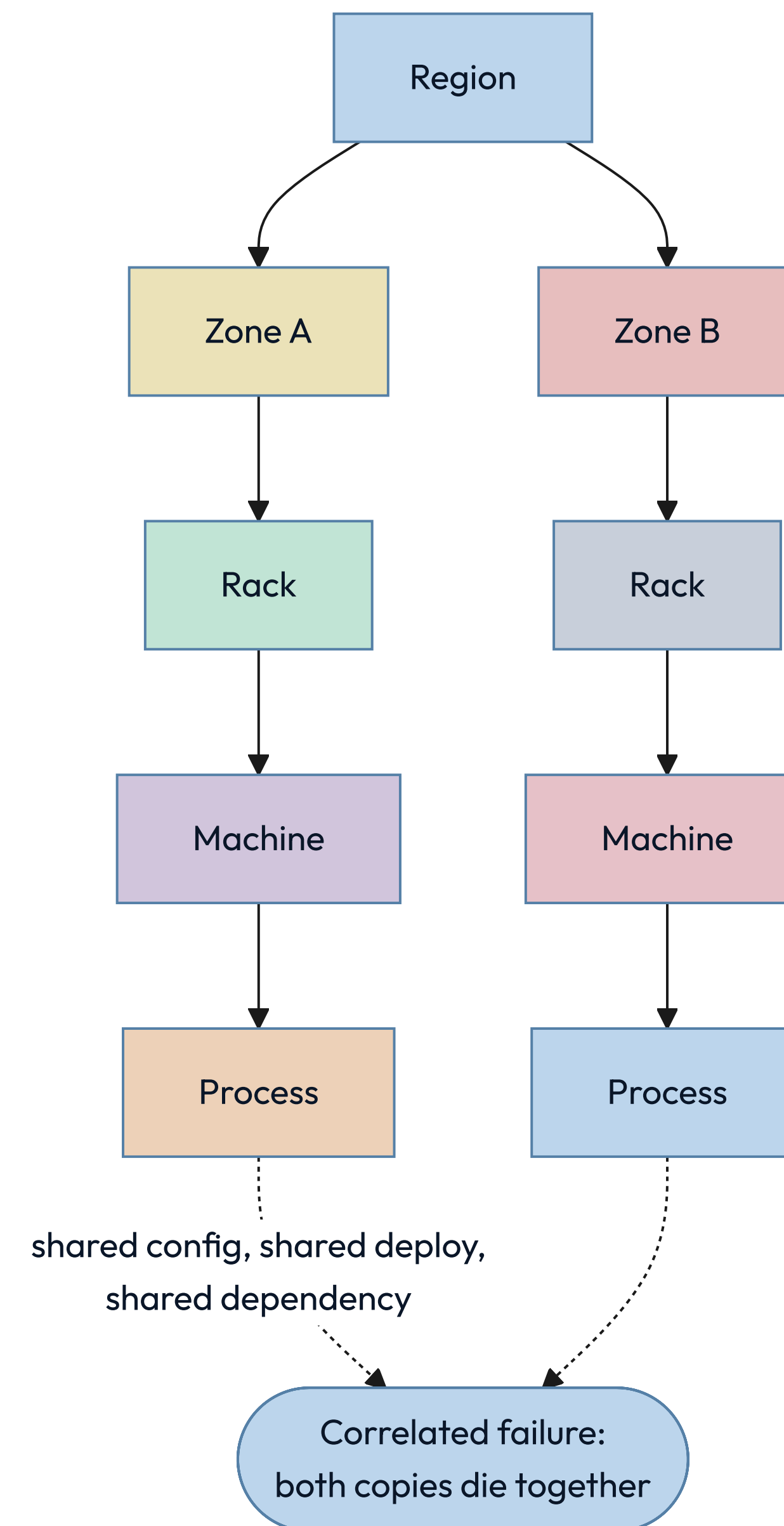
Adding a machine is a routine act

No manual steps, no rebalancing outage, no cold cache stampede.

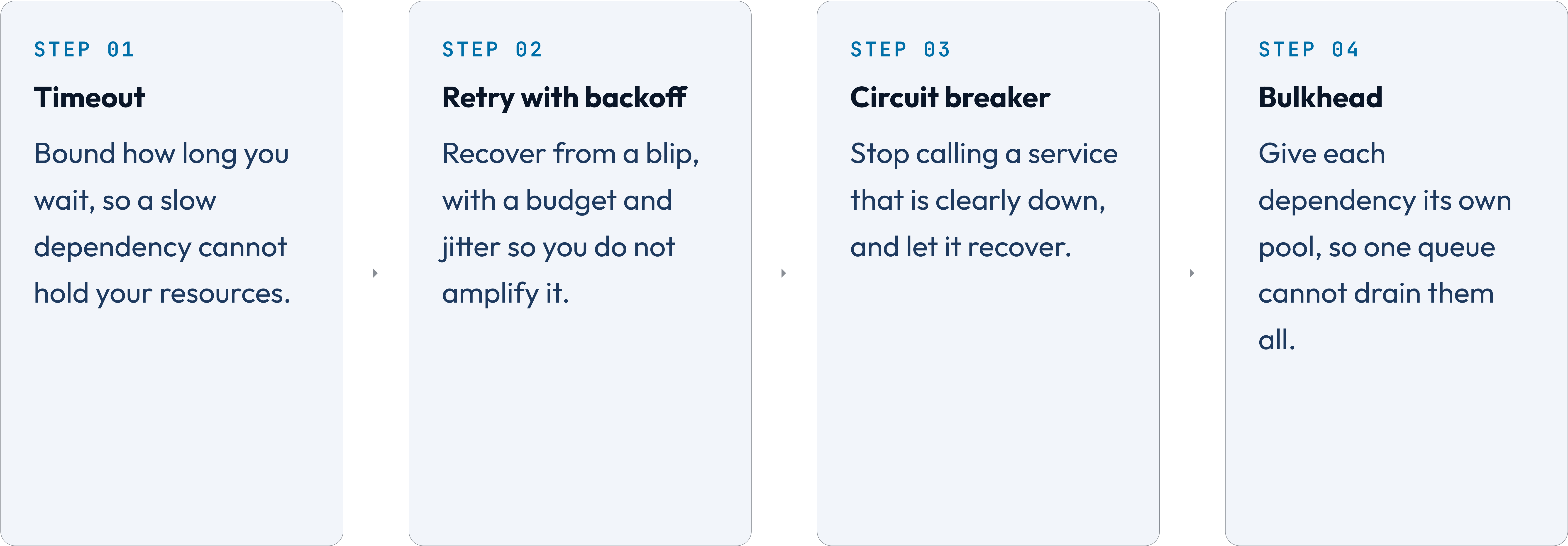
KIT FIVE

**Reliability: failure is the
environment, not an exception.**

Redundancy only helps when the copies fail independently.



Four mechanisms stop one failure from becoming every failure.



Four terms, often confused, that decide what you are allowed to ship.

01 Indicator

The SLI: the share of requests served under 300 milliseconds.

02 Objective

The SLO: your internal target, such as 99.9 percent over 28 days.

03 Agreement

The SLA: the external promise, with money attached and always looser.

04 Budget

What the objective lets you spend. Exhausted, feature work stops.

WHAT THE NINES COST

Availability targets are budgets, and the budget shrinks fast.

01	99 percent	3.65 days down per year
02	99.9 percent	8.8 hours per year
03	99.99 percent	53 minutes per year
04	99.999 percent	5.3 minutes per year

Past three nines, the humans stop being fast enough.

Fifty-three minutes a year is less than one incident with a page, a login, a look at a dashboard and a decision. Four nines therefore means automatic failover and automatic rollback, not a faster on-call engineer.

Each nine roughly multiplies cost. Pick the number the business actually needs, and write down what you are not buying.

Three signals answer three different questions; none of them replaces the others.

Metrics

Cheap, aggregate, always on.
Tells you that something is wrong.

Logs

Detailed, expensive, per event.
Tells you what happened in one case.

Traces

One request across every service.
Tells you where the time went.

KEY INSIGHT

If you cannot answer "which dependency is slow" in one minute, you have metrics but not observability.

A well-designed system gets worse in an order you chose.

What do you drop first?

Under pressure something must give. Either you decided in advance which features degrade, or the system decides at random and drops the checkout page.

YOU KNOW YOU'RE HERE WHEN

Every feature has a stated tier, and the lowest tier fails first by design.

READ PATHS OUTLIVE WRITE PATHS

Serving stale data beats serving an error page for most products.

THE FALLBACK IS TESTED

An untested degraded mode is an untested code path in your worst hour.

Trust a system in production only when these four hold.

01

Every dependency has a defined failure behavior

Written down: degrade, queue, or fail fast. Never "we will see."

02

Redundant copies fail independently

Different zones, different deploys, different upstreams. Otherwise it is one copy.

03

Recovery is practiced

Restores, failovers and rollbacks are exercised on a schedule, not improvised.

04

The system tells you before the user does

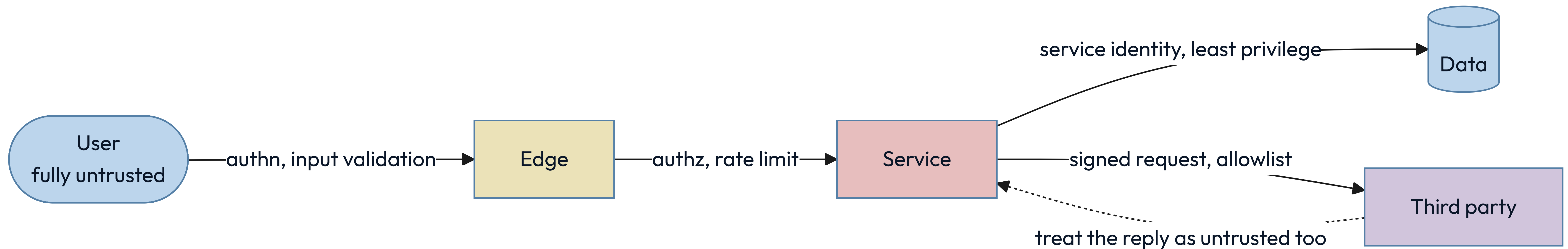
Alerts fire on the indicator, not on the complaint.

KIT SIX

**Security: assume the
boundary is already crossed.**

TRUST BOUNDARIES

Every arrow that crosses a trust boundary needs a check on the far side.



THE PAIR EVERYONE CONFLATES

Authentication asks who you are; authorization asks what you may do.

AUTHENTICATION

Establishes identity once, at the edge, and hands down a verifiable claim. Getting it wrong lets the wrong person in.



AUTHORIZATION

Decides, on every single request, whether this identity may touch this specific object. Getting it wrong lets the right person read someone else's data.

Least privilege is measured by what a stolen credential can reach.

How far does one compromise travel?

Assume any single credential will eventually leak. The design question is what an attacker holding it can do, and for how long.

YOU KNOW YOU'RE HERE WHEN

You can state the blast radius of each service account in one sentence.

SCOPE IT AND EXPIRE IT

Narrow permissions, short lifetimes, and automatic rotation beat a careful human.

SEPARATE READ FROM WRITE

Most services need one or the other, and almost none need to delete.

Encryption has three states, and a fourth thing that outranks all of them.

- 01 In transit** TLS everywhere, including between your own services. Networks are untrusted.
- 02 At rest** Disk and backup encryption. Stops a stolen device, not a stolen credential.
- 03 In use** Decrypted in memory to be processed. The hardest state, and the one attacked.
- 04 Keys** A managed store or an HSM, rotated on a schedule, never in the repository.

Six questions find most of the holes before an attacker does.

- ✓ Can someone pretend to be another identity? spoofing
- ✓ Can data be changed in transit or at rest? tampering
- ✓ Can an actor deny having done something? repudiation
- ✓ Can private data leak to the wrong reader? disclosure
- ✓ Can one caller exhaust a shared resource? denial
- ✓ Can a caller gain rights they were not granted? elevation

Sign your name to a design only if these four are true.

01

Every request is authorized against the specific object

Not the endpoint. Object-level checks are where real breaches happen.

02

Secrets never enter the repository or a log line

They live in a managed store, are injected at runtime, and rotate.

03

All input is untrusted, including from your own services

A compromised internal caller is the ordinary case, not the exotic one.

04

Every privileged action is attributable

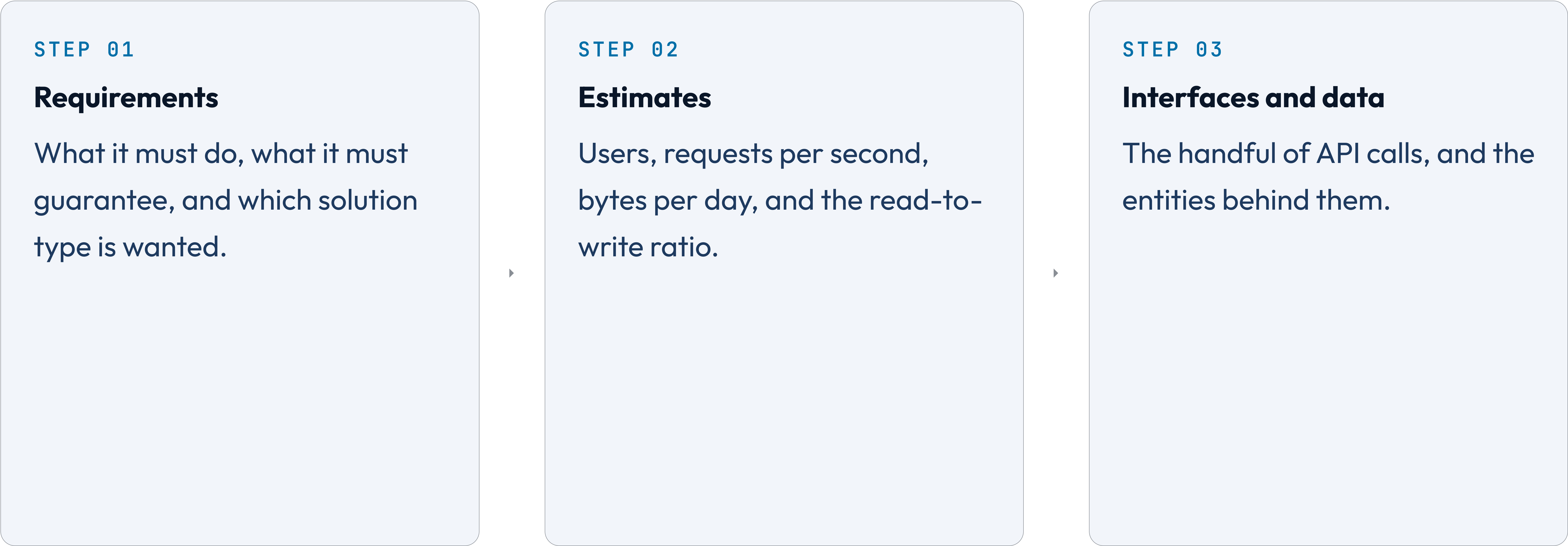
An immutable audit record naming who, what, when, and from where.

05

PART FIVE

Two systems, designed end to end.

Three steps before a single box goes on the whiteboard.



Three steps that turn the frame into a defensible design.

STEP 01

Architecture

The components, the request path, and the store behind each one.



STEP 02

Bottleneck and tradeoff

Name the scarce resource, then name the choice you made and what it cost.



STEP 03

Failure modes

What breaks first, what degrades, and which invariants must survive.

DESIGN ONE

A photo feed at global scale.

Four operations carry the whole product.

01

Post a photo

Upload media, attach a caption, publish to followers.

02

Follow an account

Build the social graph that decides whose posts you see.

03

Read a feed

A ranked, paginated list of recent posts from accounts you follow.

04

React

Like and comment, with counts visible on every post.

Four guarantees decide the architecture more than the features do.

01

The feed serves in under 200 milliseconds

Server-side, 99th percentile, measured from the reader's own region.

02

Reads dominate writes by about sixty to one

Every design decision should favor the reader.

03

Posts and media are durable forever

A lost photo or a lost follow graph is unrecoverable. Only derived data rebuilds.

04

Eventual consistency is fine on the feed

A late post and a lagging count are fine; a blank one is not.

Five numbers, and the architecture is already half decided.

01	Daily active users	500 M
02	Photos per day	100 M
03	Average writes	1.2 K per second
04	Average feed reads	70 K per second
05	Media stored daily	200 TB

The ratio, not the totals, is what the design has to answer.

Seventy thousand reads against twelve hundred writes is sixty to one — both averages, compared like for like. A 3× peak gives 200K reads and 3.5K writes; the ratio holds, the absolute numbers do not.

So: precompute the feed and pay whatever the write path costs. The storage line says something else — 200 terabytes a day of immutable media belongs in an object store, never a database.

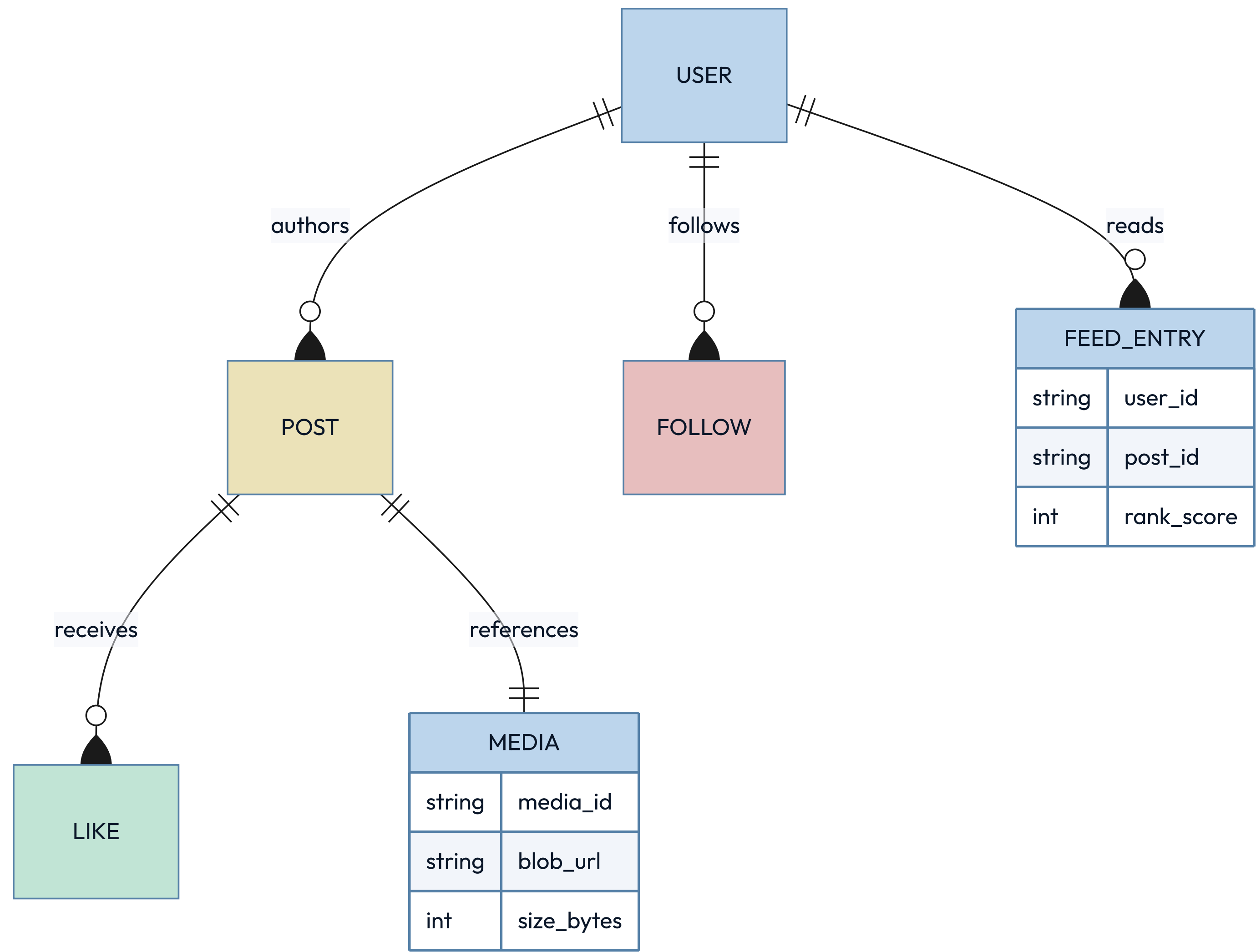
Eight endpoints, and one pagination choice that stops duplicates.

```
POST    /v1/media      -> { upload_url, media_id } # presigned, direct to blob
POST    /v1/posts      { media_id, caption }   # Idempotency-Key required
GET     /v1/feed?cursor= -> { items[], next_cursor } # cursor, never offset
GET     /v1/posts/{id} -> { post, media, counts } # single post, cache friendly
GET     /v1/users/{id}/posts?cursor= # profile grid, same cursor
POST    /v1/posts/{id}/like # idempotent by (user, post)
POST    /v1/follows     { target_id } # idempotent by (user, target)
DELETE  /v1/posts/{id}   # tombstone, feeds filter it
```

★ A cursor encodes the last item seen. An offset renumbers itself every time somebody posts, so page two repeats what page one already showed.

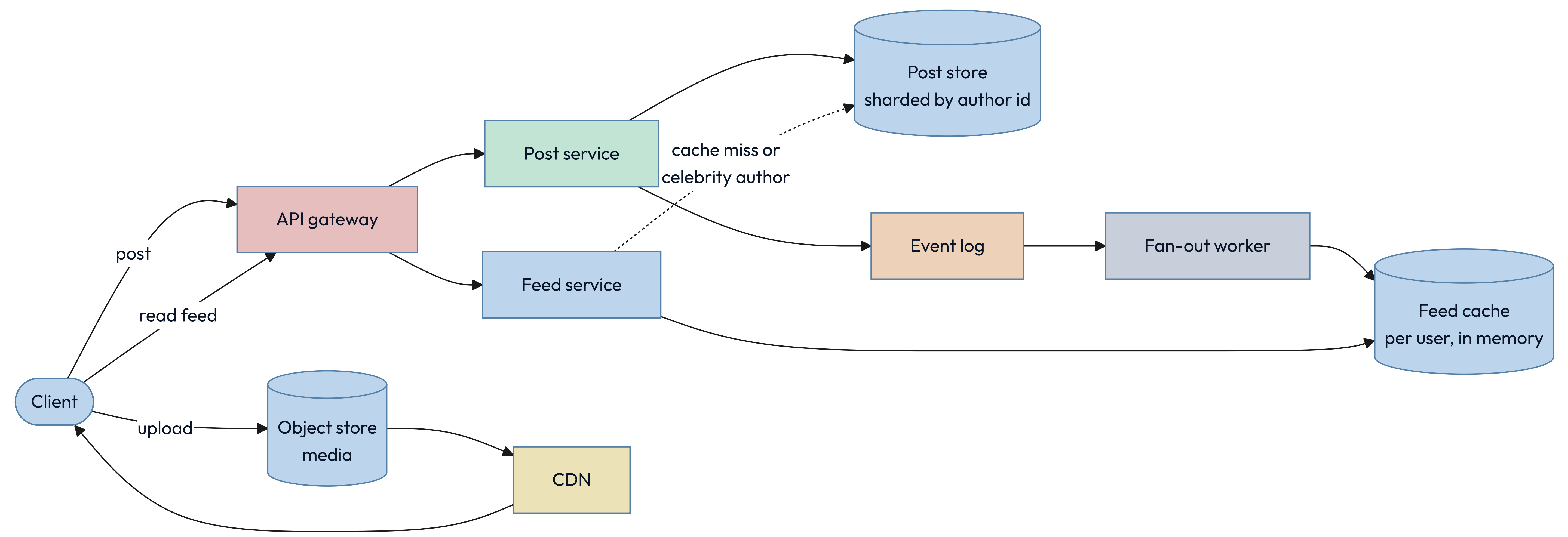
INSTAGRAM • THE DATA

Six entities, and the one you read from is not a source of truth.



★ *FEED_ENTRY is the precomputed feed, held in cache and rebuildable from POST. Everything else is stored.*

The write path does the work so the read path stays a lookup.



✦ Author id is the shard key because every read that reaches the post store asks for one author's recent posts. Post ids embed the author, so a single-post lookup still routes in one hop.

Fan-out on write and fan-out on read fail at opposite ends of one graph.

FAN-OUT ON WRITE

Push each post into every follower's feed when it is published. Reads become one lookup, and a celebrity turns one post into 50 million writes.



FAN-OUT ON READ

Assemble the feed at request time from the accounts you follow. Writes are trivial, and every read now costs hundreds of queries.

DECISION

Use both, and let follower count choose between them.

The two strategies fail in opposite directions, so the design uses each where the other breaks.

PICK ONE STRATEGY FOR EVERYONE

- Simple to build and simple to explain. It is also guaranteed to fail at one end: write fan-out drowns on celebrities, read fan-out blows the latency budget for everyone.

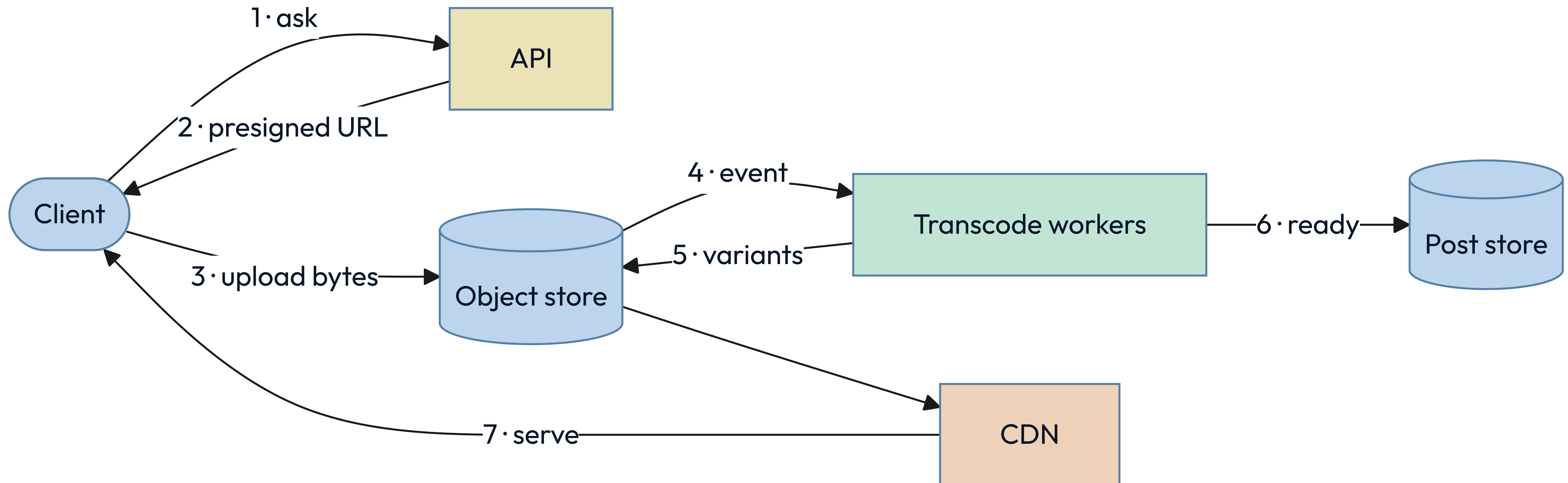
HYBRID, SPLIT AT A FOLLOWER THRESHOLD

- Push from ordinary accounts, pull from high-follower accounts, merge at read time. Two code paths instead of one, and the threshold becomes a tuning knob you own forever.

RECOMMENDATION

Fan out on write below the threshold, pull at read above it, and merge in the feed service.

Media never touches your API servers, in either direction.



Four failures are certain, and each has a designed answer.

Celebrity post

Fan-out queue floods.
Answer: the read-time pull path above the threshold.

Feed cache eviction

Cold users cost a rebuild. Answer: rebuild from the post store, lazily.

Transcode backlog

Uploads outpace workers. Answer: bounded queue, serve the original meanwhile.

Region loss

A whole zone disappears. Answer: media replicated, feeds rebuildable from source.

KEY INSIGHT

Every answer here is either a queue with a bound or a store that can be rebuilt.

Five sentences hold, or the design is not finished.

- ✓ A published post reaches every follower's feed. eventually
 - ✓ Media is never lost once an upload is acknowledged. durable
 - ✓ A like counts exactly once per user and post. idempotent
 - ✓ A feed page never repeats or skips an item. cursor
 - ✓ Every derived store rebuilds from the post store. rebuildable

DESIGN TWO

A conversational model service.

WHY THIS ONE IS DIFFERENT

The scarce resource is not the database. It is attached memory on a GPU.

Every design so far treated storage and network as the limits. Model serving moves the bottleneck: a request holds a slice of GPU memory for the entire time it is generating, and generation is slow on purpose because the user reads it as it arrives.

That single fact reshapes queuing, batching, caching and cost.

Four operations, and the second one is the whole product.

01

Send a message

A prompt plus the conversation so far, returning a generated reply.

02

Stream the reply

Tokens arrive as they are produced, not in one block at the end.

03

Keep conversation history

Prior turns are retrievable and become part of the next request's context.

04

Ground answers in documents

Retrieve relevant text at request time so answers cite current facts.

Four guarantees, and the first two are in direct conflict.

01

First token in under 500 milliseconds

Perceived speed is time to first token, not time to the last one.

02

Cost per million tokens must fall every quarter

Which means large batches, which means waiting, which fights the line above.

03

No prompt or reply leaks between users

A shared cache keyed carelessly is a data breach, not a bug.

04

Degrade rather than refuse

Under load, route to a smaller model before returning an error.

Five numbers, all order-of-magnitude, all decisive.

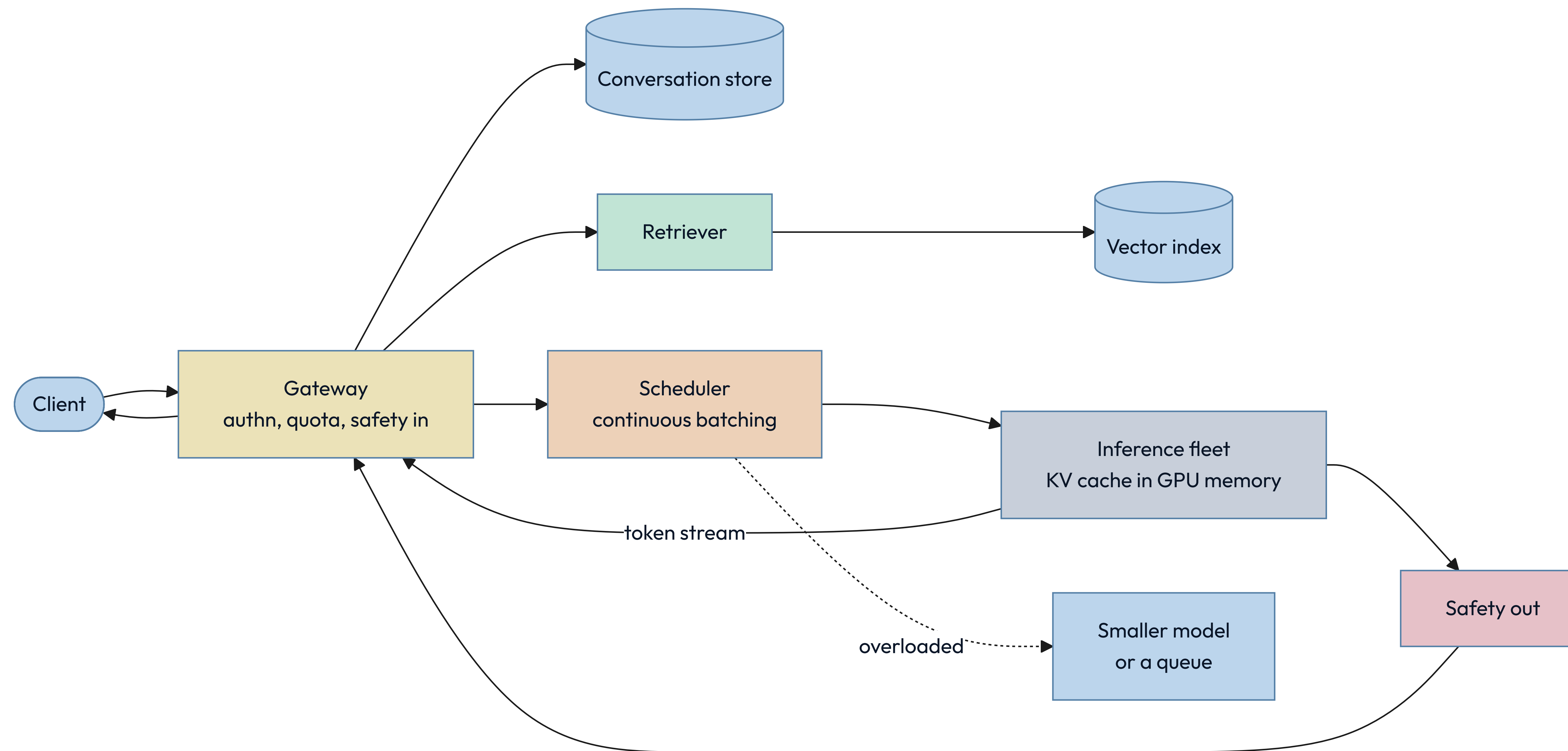
01	Messages per day	1 B
02	Peak requests	40 K per second
03	Tokens per reply	500
04	Seconds per slot	10 s
05	Concurrent streams	400 K

Little's Law turns a request rate into a hardware order.

Forty thousand requests per second, each holding a slot for ten seconds, is four hundred thousand concurrent generations. Divide by the sequences one accelerator holds in memory, and you have the fleet size before choosing a framework.

Little's Law also says which lever shrinks it. Halve the tokens per reply and you halve the fleet. Halve the time per token and you do not — decode speed is set by memory bandwidth, so buying it back costs the accelerators you were saving.

Two stores, one scheduler, and a fleet that is always the constraint.



Generation is memory-bound, which is why batching is the only real lever.

Why is one request per accelerator wasteful?

Producing one token reads the entire model from memory and does very little arithmetic with it. The hardware sits idle waiting on memory unless many sequences share that read.

YOU KNOW YOU'RE HERE WHEN

Utilization is low while latency is high, and adding machines barely helps.

BATCHING AMORTIZES THE READ

Sixty-four sequences share one weight read, so throughput rises almost linearly.

THE KV CACHE IS THE CEILING

Each sequence holds attention state for every token, and that memory is the limit.

Bigger batches make it cheaper for everyone and slower for each person.

LARGE BATCHES

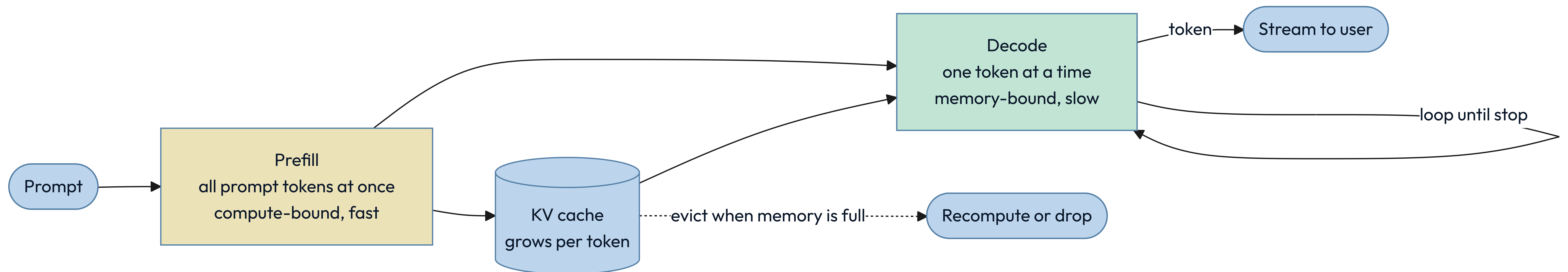
More sequences per weight read, so cost per token falls and the fleet shrinks. Each request waits longer to join a batch and generates more slowly inside it.



SMALL BATCHES

First token arrives fast and generation feels responsive. The same traffic now needs several times the hardware, and cost per token rises with it.

Two phases with opposite shapes hide inside one request.



Four caches, and only the first is shareable without a per-user key.

Prefix cache

Reuses the KV state of a public system prompt. A large win; a hit is faster, so scope it per tenant.

Conversation KV

Keeps a live chat's state warm between turns. Per user, never shared.

Retrieval cache

Caches document lookups. Shared only across users with identical access.

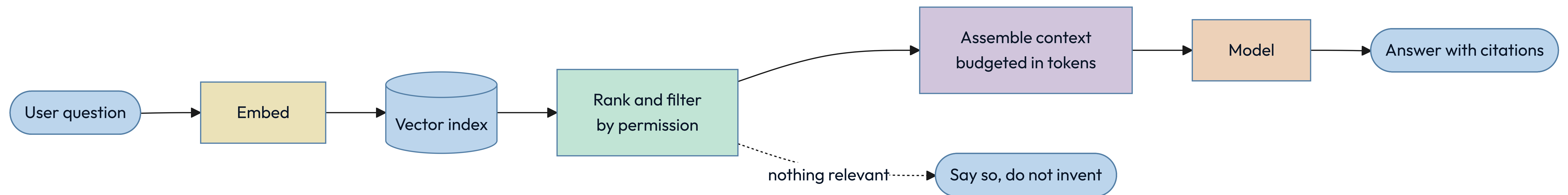
Response cache

Caches whole answers. Tempting, and the usual route to a cross-user leak.

KEY INSIGHT

A cache key that omits the identity is how private text reaches a stranger.

Retrieval puts current facts in the context the weights never learned.



Safety is three checks in three places, not one filter in the middle.

On input

Classify the request before spending a GPU-second on it.

On output

Check the generated text before the user sees it, streaming in chunks.

On action

Any tool the model can call is authorized as the user, never as the service.

KEY INSIGHT

The output check must work on a stream, because the user is already reading.

The service is not safe to run unless these five hold.

- ✓ No user's text reaches another user's context. isolation
- ✓ Every cache key names the identity that may read it. scoped
- ✓ A tool call carries the user's rights, not the service's. delegated
- ✓ Every request has a token budget and a deadline. bounded
- ✓ Overload degrades to a smaller model, never an error. graceful

The method is identical; only the scarce resource changes.

DIMENSION	PHOTO FEED	CHAT SERVICE
Scarce resource	Read throughput	GPU memory
Central tradeoff	Fan-out on write or read	Batch size against latency
Consistency	Eventual on the feed	Strict per conversation
Degrades by	Serving a stale feed	Routing to a smaller model
Cost driver	Storage and egress	Accelerator hours

06

PART SIX

What to do with this on Monday.

THE PRACTICE

Four habits turn a reference into judgment.

STEP 01

Model one a week

Draw a system you use. Name its bottleneck.

STEP 02

Say the type out loud

Name the rung before you design anything.

STEP 03

Invariants first

Three sentences that must never go false.

STEP 04

Estimate, then argue

Two numbers end most debates in a minute.

THE REVIEW

Six questions to ask of any design, including your own.

- ✓ Which solution type is this, and does everyone agree? frame
- ✓ What is the bottleneck, and what number proves it? evidence
- ✓ Which invariants must never go false? correctness
- ✓ What breaks first under ten times the load? scale
- ✓ What happens when each dependency is slow rather than down? failure
- ✓ What can one stolen credential reach? security

Four questions expose more than an hour of diagrams.

01 What did this drawing leave out on purpose?

No answer means a picture, not a model.

02 Which resource runs out first, at what number?

No number means it was never sized.

03 What does a user see during a partition?

This forces the consistency choice open.

04 How would you rebuild this from the source?

A store you cannot rebuild fails silently.

THE PRINCIPLES

Five sentences worth keeping longer than any of the kits.

01

Every complex system that works evolved from a simple one that worked. None was ever designed complex from scratch.

02

A model is defined by what it leaves out, and an unstated omission is a bug you have not found.

03

Constraints are the good news: they turn infinite answers into a choice you can defend.

04

Name the solution type before the architecture, or you will design a beautiful answer to another question.

05

At any moment one constraint binds, and everything else is decoration until that one moves.

Glossary, A to C

A – C

TERM	DEFINITION
ACID	The four transaction guarantees: all-or-nothing, rules hold, no visible interleaving, survives a crash.
API	The contract one system offers another: the operations and what they promise.
Backpressure	A slow consumer telling a fast producer to stop, instead of silently queueing forever.
BASE	Accept that replicas diverge and converge later, paying in application complexity.
Blast radius	Everything that stops working when one component, credential or zone fails.
Bottleneck	The one resource that limits the whole. Improving anything else changes nothing.
CAP	Split by a network fault, a system must answer with stale data or refuse.
CDN	Caches placed near readers so bytes travel a short distance.
CQRS	Read from a store shaped for reading, write to one shaped for writing.

Glossary, C to I

C - I

TERM	DEFINITION
CRUD	Create, read, update, delete: the smallest useful vocabulary for a data model.
DNS	The global lookup turning a name a human types into an address.
Fan-out	One event becoming many writes or many reads, depending on where you pay.
GPU	A processor with fast attached memory, used for the matrix work inference is made of.
HSM	A tamper-resistant device that signs on request, so the key never leaves it.
Idempotent	Doing it twice has the same effect as doing it once. What makes a retry safe.
Invariant	A claim that must never go false, during a deploy, a partition or a migration.
IOPS	Separate reads or writes per second, usually the limit before raw bandwidth.

Glossary, J to R

J – R

TERM	DEFINITION
JWT	A signed bundle of claims a client carries, checkable without a database lookup.
MVP	The smallest build that puts a real answer in front of a real user.
OLAP	Query work that scans many rows and few columns to answer an aggregate question.
OLTP	Query work that touches few rows across many columns for one user action.
PACELC	Partitioned, choose availability or consistency; otherwise, latency or consistency.
Partition	A network split that leaves nodes unable to reach each other, but still running.
RAG	Fetching documents at request time so answers cite facts the weights never learned.
REST	Resources acted on by a small fixed set of verbs, which is what makes responses cacheable.

Glossary, S to W

S – W

TERM	DEFINITION
Shard	One slice of a partitioned dataset. The shard key decides which questions stay cheap.
SLI, SLO, SLA	The measurement, the internal target held against it, and the external promise.
Tail latency	What the slowest few percent of requests experience. Where users actually live.
TLS	Encrypts and authenticates a connection, so an untrusted network cannot read it.
TTL	How long a cached or replicated copy may be served before it must be refreshed.
WAL	An append-only record written before a change, so a crash replays forward.
Write amplification	One logical write becoming several physical ones: compaction, erase blocks, secondary indexes.

SYSTEMS, AND HOW TO DESIGN THEM

Draw the boundary, name
the constraint, state the
invariant. Then design.

*The kits will age. The seven words and the six
questions will not — they are how you will read
whatever replaces the technology in them.*

